



 Latest updates: <https://dl.acm.org/doi/10.1145/3290323>

RESEARCH-ARTICLE

Fully abstract module compilation

KARL CRARY, Carnegie Mellon University, Pittsburgh, PA, United States

Open Access Support provided by:

Carnegie Mellon University



PDF Download
3290323.pdf
04 March 2026
Total Citations: 5
Total Downloads: 920



Published: 02 January 2019

[Citation in BibTeX format](#)



Fully Abstract Module Compilation

KARL CRARY, Carnegie Mellon University, USA

We give a translation suitable for compilation of modern module calculi supporting sealing, generativity, translucent signatures, applicative functors, higher-order functors and/or first-class modules. Ours is the first module-compilation translation with a dynamic correctness theorem. The theorem states that the translation produces target terms that are contextually equivalent to the source, in an appropriate sense. A corollary of the theorem is that the translation is fully abstract. Consequently, the translation preserves all abstraction present in the source. In passing, we also show that modules are a definitional extension of the underlying core language. All of our proofs are formalized in Coq.

CCS Concepts: • **Theory of computation** → **Logic and verification**; • **Software and its engineering** → **Modules / packages**;

Additional Key Words and Phrases: Modules, full abstraction, phase separation

ACM Reference Format:

Karl Crary. 2019. Fully Abstract Module Compilation. *Proc. ACM Program. Lang.* 3, POPL, Article 10 (January 2019), 29 pages. <https://doi.org/10.1145/3290323>

1 INTRODUCTION

ML-style module calculi [Harper et al. 1986] provide powerful abstraction tools for various programming-in-the-large settings. Structures collect related type and term components, as well as other structures, together in a single expression. More than just a namespace management mechanism, modules can be made opaque, so that clients can use them only through their specified interfaces. Functors map modules to other modules, which allows the programmer to give an implementation of an abstract data type that depends parametrically on an unknown implementation of another.

In an untyped setting, compiling modules is not too difficult. One can simply erase the types and compile modular code as ordinary code, rendering modules as tuples or records, and functors and functions. Nevertheless, there are compelling reasons to prefer type-preserving compilation. Chief among them is the assistance that type structure offers in achieving compiler correctness. Simply as a matter of engineering, experience shows that it is very difficult to implement (accidentally) a type-preserving global transformation¹ that is incorrect and yet type correct.

Types can take on an even more significant role in compiler verification. The best correctness result for a compiler phase is *contextual equivalence*, which says that the source and translated terms are indistinguishable by any legal observation (in an appropriate sense). This is the most robust form of translation correctness because it applies regardless of how the translated code is used, be it translated further (vertical composition), linked with other code (horizontal composition), or used in other way. Types are essential to contextual equivalence: without type structure to circumscribe

¹We refer here to such transformations as CPS or closure conversion, in contrast to *local* transformations such as arithmetic optimization, where types help much less.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2019 Association for Computing Machinery.

2475-1421/2019/1-ART10

<https://doi.org/10.1145/3290323>

how the code may be used, too many observations are possible, making non-trivial contextual equivalence nearly impossible.

In this paper we give the first compilation strategy for ML-style modules that (1) supports the full array of abstraction mechanisms offered by the Standard ML module system (*i.e.*, structures, sealing, generative functors, and translucent signatures) [Milner et al. 1990] as well as some other important features (applicative functors, higher-order functors, and first-class modules), and (2) admits a contextual equivalence result between modules and their translation.

As a corollary of contextual equivalence, the module translation is *fully abstract*: Modules are equivalent exactly when their translations are equivalent. The left-to-right portion of this is also called *abstraction preservation*. It says that no legal target code can break any abstractions present in the source code.

This is particularly valuable today, as functional languages are increasingly becoming part of heterogeneous (multi-language) development environments. Full abstraction means that a programmer can be confident that his or her abstractions will be respected even by code linked in from another language, provided types are preserved and enforced [Patrignani et al. 2019].

Moreover, our compilation strategy admits *separate compilation*, meaning that units can be linked entirely in the target language, without reference to the source code that generated them. In practice, this means that every module expression—including functors—is independently translated to a target-language expression.

The first mathematical examination of the compilation of modules was undertaken by Harper, *et al.* [1990]. Their interest was not in compilation *per se*, but in the equational theory of higher-order modules. Specifically, they wished to show that code that employs functors respects a compile-time/run-time *phase distinction*, meaning that code can always be type-checked without executing any code.

That the phase distinction is respected is not obvious *a priori*. A module contains both type and term components, and a functor is a function that maps modules to modules, so at first glance it appears as though the type components of the result can depend on the term components of the argument. If so, it would be necessary to execute the argument terms in order to determine the result types.

This does not happen because the module language is structured in such a way that types can never depend on terms. Harper, *et al.* show that module expressions can be separated into two parts: a static (or “compile-time”) phase and a dynamic (or “run-time”) phase. Dynamic-phase expressions can depend on static-phase expressions, but not vice versa. This relies crucially on modules being second-class.

Harper, *et al.* go on to show that the static-phase equational theory is unaffected by the deletion of all dynamic-phase equations. But for our purposes, we are interested in their phase-separation algorithm itself. In their algorithm, every module M is split into two pieces $[c, e]$, in which c is a type constructor representing M ’s static portion, and e is a term representing its dynamic portion. Both pieces can be rendered without reference to the module language, so modules are compiled away.

Harper, *et al.*’s module language supported some important features, but it omitted other ones, most notably sealing and generativity. The only form of abstraction available was lambda abstraction, which meant that their language supported only client-side abstraction, not provider-side abstraction, which is more robust [Crary 2017].

Subsequently, Shao [1998] employed a similar algorithm in a compiler for Standard ML. Unlike Harper, *et al.*, Shao’s source language supports sealing and generativity. However, the translation

deals with them simply by removing them, so it does not preserve abstraction. (This was consistent with his purpose of facilitating cross-module inlining.)

Next, Shan [2006] and Rossberg, *et al.* [2014] proposed a different approach for compiling modules with sealing and generativity, in which sealing was treated as introducing an existential type. This provides a very natural treatment of abstraction [Mitchell and Plotkin 1988].

Neither Shan nor Rossberg, *et al.* included any dynamic correctness result, much less full abstraction. In fact, neither one gave an evaluation semantics for the source language.² We resolve the question here in the negative; their translation is *probably not* fully abstract. More precisely, their translation is not fully abstract if the target’s dynamic semantics provides any strict mechanism (e.g., call-by-value or seq) capable of observing the termination of a function expression. However, the problem is easily corrected by a change to the target’s static semantics.

This paper’s main contribution is the first module-compilation algorithm with a dynamic correctness proof. The algorithm is given as a phase-separation translation: as in Harper, *et al.*’s original translation, we split each pure module into a type constructor and a term. The correctness result is as strong as possible: we show that modules are translated to contextually equivalent terms, and we conclude, as a corollary, that the translation is fully abstract.

The most obvious obstacle to the contextual equivalence result is that the source expression and its translation belong to different types, different syntactic classes, and indeed different languages. The difference in languages is easily dealt with: our target language is a strict subset of our source language, obtained by omitting all module-oriented forms. Thus, we can compare a source expression and its translation simply by taking the translation as another source expression.

To mediate the difference in types and syntactic classes, we show that we can define a pair of functions Snd and Join that convert back-and-forth between modules and terms of appropriate types. The result then states that if a module M is phase-separated into static portion c and dynamic portion e , then $\text{Join}[c, e]$ is contextually equivalent to M . Since Snd and Join are inverses, one may equivalently say that $\text{Snd}M$ is contextually equivalent to e .

If Join merely traversed e to reverse the phase-separation translation, this result would be trivial. It does not. The important fact about Snd and Join is they are defined *within the grammar of the language*. They are not able to traverse the syntax of an expression, and thus they can only interact with an expression through the interface given by its type. Their purpose is to coerce between interfaces.

One observation that we can draw from this result is that modules are a definitional extension of the core language. Put less formally, the module language can be viewed as merely a very convenient syntax for writing modular programs. A practical consequence of this observation is that some useful module features (notably first-class modules [Russo 2000]) need not be included as distinct features to be compiled, because they are already definitional extensions of the language.

Our development proceeds as follows: First, we summarize our module calculus and state some preliminary results regarding it (Section 2). Our module calculus is nearly identical to Cray [2017], except we add a value restriction for polymorphic functions. Second, we give the phase separation algorithm (Section 3). Third, we define contextual equivalence (Section 4). This is adapted from Cray, with small changes to deal with the value restriction. We also state some useful lemmas about contextual equivalence, corollaries of the development of logical equivalence in Cray. Finally we prove our correctness results (Section 5): We define Snd and Join, show that they are inverses,

²This was understandable in both cases. Shan was working from a pre-existing type theory [Dreyer *et al.* 2003] that did not commit to a specific dynamic semantics. Rossberg, *et al.* were *defining* a module system by giving an elaboration of ML modules into a module-free core language, so the source language necessarily had no independent semantics at all. (Nevertheless, the question remains whether the elaboration realizes the programmer’s expectations.)

$k ::= 1$	unit kind			
$ T$	types			
$ S(c)$	singleton kind	$M ::= m$		
$ \Pi \alpha:k.k$	dependent functions	$ \star$	unit module	
$ \Sigma \alpha:k.k$	dependent pairs	$ \langle c \rangle$	atomic module	
$c, ::= \alpha$		$ \langle e \rangle$	atomic module	
$\tau \quad \star$	unit constructor	$ \lambda^{\text{gn}} \alpha/m:\sigma.M$	generative functor	
$ \lambda \alpha:k.c \mid c \ c$	lambda, application	$ M \ M$	generative app.	
$ \langle c, c \rangle$	pair	$ \lambda^{\text{ap}} \alpha/m:\sigma.M$	applicative functor	
$ \pi_1 c \mid \pi_2 c$	projection	$ M \cdot M$	applicative app.	
$ \text{unit}$	unit type	$ \langle M, M \rangle$	pair	
$ \tau_1 \rightarrow \tau_2$	functions	$ \pi_1 M \mid \pi_2 M$	projection	
$ \tau_1 \times \tau_2$	products	$ \text{unpack } [\alpha, x] = e \text{ in } (M : \sigma)$	unpack	
$ \forall \alpha:k.\tau$	universals	$ \text{let } x = e \text{ in } M$	term binding	
$ \exists \alpha:k.\tau$	existentials	$ \text{let } \alpha/m = M \text{ in } (M : \sigma)$	module binding	
$e ::= x$		$ M :> \sigma$	sealing	
$ \star$	unit term	$\sigma ::= 1$	unit signature	
$ \lambda x:\tau.e \mid e \ e$	lambda, application	$ \langle k \rangle$	atomic signature	
$ \langle e, e \rangle$	pair	$ \langle \tau \rangle$	atomic signature	
$ \pi_1 e \mid \pi_2 e$	projection	$ \Pi^{\text{gn}} \alpha:\sigma.\sigma$	generative functors	
$ \Lambda \alpha:k.e$	polymorphic fun.	$ \Pi^{\text{ap}} \alpha:\sigma.\sigma$	applicative functors	
$ e[c]$	polymorphic app.	$ \Sigma \alpha:\sigma.\sigma$	pairs	
$ \text{pack } [c, e] \text{ as } \exists \alpha:k.\tau$	existential package	$\Gamma ::= \epsilon$	empty context	
$ \text{unpack } [\alpha, x] = e \text{ in } e$	unpack	$ \Gamma, \alpha:k$	constr. hypothesis	
$ \text{fix}_\tau e$	recursion	$ \Gamma, x:\tau$	term hypothesis	
$ \text{let } x = e \text{ in } e$	term binding	$ \Gamma, \alpha/m:\sigma$	module hypothesis	
$ \text{let } \alpha/m = M \text{ in } e$	module binding			
$ \text{Ext } M$	extraction			

Fig. 1. Syntax

show that modules translate to contextually equivalent terms (modulo Join), and conclude full abstraction as a corollary.

All of our proofs are formalized in Coq, and may be found at:

www.cs.cmu.edu/~crary/papers/2018/famc-formal.tgz

2 THE MODULE CALCULUS

Our module calculus is nearly identical to Crary [2017], which is adapted from that of Dreyer [2005], which in turn builds on a long line of prior work on module calculi [Dreyer et al. 2003; Harper and Lillibridge 1994; Harper et al. 1990; Harper and Stone 2000; Leroy 1994, 1995; MacQueen 1986; MacQueen and Tofte 1994; Russo 1998; Shao 1999; Stone and Harper 2000]. The sole change from Crary is the addition of a value restriction to polymorphic functions. (As we will show, the value restriction is necessary for our translation to be fully abstract.) The module calculus itself is not a research contribution of this paper; our purpose here is just to lay the foundation for our phase-separation algorithm.

The module calculus is intended to account for the fundamental elements of a module system that are vital for data abstraction [Crary 2017]. However, it does not account for all of the convenience features of modern module systems: Fields of modules are identified by position, not by name. Coercion of a module to a specified signature (*i.e.*, by dropping fields, reordering fields, or monomorphizing polymorphic functions) is not done automatically; to do so creates complications that are orthogonal to the abstraction concerns we focus on here. Like nearly all module calculi (but

not Harper and Mitchell [1993]), sharing specifications are not primitive, but are taken as syntactic sugar over translucent signatures. The open declaration, which is useful in practice but makes little sense in a type theory with a proper notion of binding, is not supported. First-class modules (supported in some dialects of ML [Russo 2000]) are not explicitly supported (but see Section 6). We also do not support recursive modules [Crary et al. 1999; Dreyer 2005; Russo 2001] (supported in some dialects of ML [Leroy 2003; Russo 2001]); the abstraction implications of recursive modules are not yet well examined. All of these features can be implemented by elaboration [Dreyer 2005; Dreyer et al. 2003; Harper and Stone 2000], but those elaborated aspects of the language fall outside the results we prove here.

Most significantly, we follow earlier work [Dreyer et al. 2003; Harper and Stone 2000; Rossberg et al. 2014] by using elaboration to address the avoidance problem [Dreyer et al. 2003], in which the structure of a program mandates that type variables leave scope, but there is no best way for a module’s type to avoid those variables. This arises most importantly in sealed arguments to functors. No satisfactory type-theoretic treatment of the avoidance problem has yet been found. The elaboration involves retaining in “hidden” modules the type variables that are supposed to leave scope. Unfortunately, the hidden-ness of those modules is not enforceable after elaboration, which may lead to tangible compromises in data abstraction. To avoid this, the programmer must manually ascribe signatures whenever type variables leave scope. This preserves abstraction, but is inconvenient.

Syntax. A key design goal of the module calculus is to respect the *phase distinction* [Harper et al. 1990], which means that the meaning of static phrases, and specifically their equivalence, can be determined without referring to dynamic phrases. The static syntactic classes are kinds (k), type constructors (c), and signatures (σ). The dynamic syntactic classes are terms (e) and modules (M). Signatures serve as the types of modules, and kinds serve (as usual) as the types of type constructors. The full syntax is given in Figure 1. (Figures 1, 2, and 4 are borrowed from Crary [2017].)

Static phrases. The constructor and kind expressions are the singleton kind calculus of Stone and Harper [2006], with some minor extensions. Actual types belong to the kind T . (We often use the metavariable τ instead of c for actual types.) The unit constructor $\star$ belongs to the kind 1 .

If τ is a type, we may form the singleton kind $S(\tau)$, which contains precisely τ (and other types equivalent to it). This is used to model type definitions and sharing specifications in signatures. Thus, if $\tau : S(\tau')$, we can conclude that τ and τ' are equivalent types. As a consequence, equivalence is context sensitive: α and τ are equivalent under the assumption $\alpha : S(\tau)$, but not under $\alpha : T$. As a further consequence, equivalence depends on the kind at which constructors are compared: $\lambda\alpha:T.\alpha$ and $\lambda\alpha:T.\tau$ are equivalent at $S(\tau) \rightarrow T$ but not at $T \rightarrow T$. Although the singleton kind primitive is restricted to types, one can use it to define singletons at any kind.

Since singletons allow kinds to depend on constructors, it is useful to support dependent kinds. The dependent form of function kinds is written $\Pi\alpha:k.k'$ and the dependent form of products is written $\Sigma\alpha:k.k'$. When α does not appear free in k' , these are written using $k \rightarrow k'$ and $k \times k'$ respectively, as usual.

Dynamic phrases. The syntax for terms is largely standard. The recursion form $\text{fix}_\tau e$ has the type τ , provided e has the type $(\text{unit} \rightarrow \tau) \rightarrow \tau$, and we evaluate $\text{fix}_\tau e$ to $e(\lambda_:\text{unit}.\text{fix}_\tau e)$.³

The module language contains static and dynamic atomic modules, generative and applicative functors, and pairs. An atomic module models a single field of a module: a static atom ($\llbracket c \rrbracket$) contains a single constructor, and a dynamic atom ($\llbracket e \rrbracket$) contains a single term. For example, the ML module:

³Observe that the argument to e is a value, so this evaluation behaves acceptably in a call-by-value setting. That would not be the case with the evaluation to $e(\text{fix}_\tau e)$ that would be implied by a more conventional typing.

```

struct
  type t = bool
  val x = true
end

```

could be elaborated $\langle \langle \text{bool} \rangle, \langle \text{true} \rangle \rangle$. (Observe that, as noted above, the field names disappear when a module is elaborated.)

Abstraction is introduced using the form $M :> \sigma$, which *seals* M so that its components can be accessed only through the interface given by σ . In particular, if a signature gives the kind of a type field as T , that type is unknown and therefore abstract. Proper enforcement of type abstraction requires that sealing be viewed as a computational effect [Dreyer et al. 2003], and that types cannot be extracted from impure modules. Thus, to use the types from a sealed module, one must bind that module to a variable; variables are always pure.

We syntactically distinguish between generative and applicative functors. A generative functor can have sealing in its body, and calling it induces an effect. In contrast, an applicative functor cannot, and does not.

For technical reasons related to the avoidance problem [Dreyer 2005, section 4.2.6], a module that either let-binds a module or unpacks an existential type must include an explicit signature annotation, and such modules are deemed to induce an effect.

Static portions, twinned variables, and phase separation. To extract a term from a dynamic atom, one uses the form $\text{Ext } M$. However, to extract a type constructor from a static atom, we employ a judgement rather than a syntactic form. The judgement $\text{Fst}(M) \gg c$ says that c is the *static portion* of M .⁴ In particular, $\text{Fst}(\langle c \rangle) \gg c$.

The reason for this design, invented by Dreyer [2005], is it removes any syntactic dependency of static phrases (such as types) on terms or modules. By disentangling the singleton kind calculus from all dynamic phrases, we are able to use Stone and Harper’s singleton-kind metatheory [Stone and Harper 2006] off the shelf.

For static extraction to work compositionally, we must be able to compute the static portion of any pure module, not only static atom literals. (Impure modules do not have static portions.) The rules for doing so are given in Figure 2. Two subtle points arise: First, by its nature, a generative functor returns nothing that is statically predictable. Therefore the static component of a generative functor is trivial. Second, for any module variable m , we must be able to compute m ’s static portion, but of course that cannot be known, so we must have a type-constructor variable prepared to stand in for m ’s static portion. This leads to the concept of *twinned variables* [Lee et al. 2007]: every module variable m is twinned with a constructor variable α , written α/m , where α is the static portion of m .⁵ However, when α is not used, we will often leave it out.

Twinned variables are recorded in the context and static portions depend on them, so the static portion judgement is written in its most general form as $\Gamma \vdash M \gg c$. However, when Γ is empty or can be determined from context, we often write $\text{Fst}(M) \gg c$, or even just $\text{Fst}(M)$ for the unique c such that $\text{Fst}(M) \gg c$.

Our static-portion rules are very similar in spirit to the *non-standard equational rules* of Harper, et al. [1990]. As in this work, Harper et al. gave rules for determining the static portions of modules as part of their static semantics, and also—*separately*—gave an algorithm for compiling modules

⁴The name Fst is motivated by the connection to phase separation, which explicitly renders modules as a pair of a static and dynamic component.

⁵An alternative [Dreyer 2005; Harper et al. 1990] is to use a naming convention to associate module and constructor variables, but this complicates binding and substitution. Sorting out those complications carefully leaves you with something very close to twinned variables.

$\text{Fst}(\sigma)$	:	kind
$\text{Fst}(1)$	$\stackrel{\text{def}}{=}$	1
$\text{Fst}(\langle k \rangle)$	$\stackrel{\text{def}}{=}$	k
$\text{Fst}(\langle \tau \rangle)$	$\stackrel{\text{def}}{=}$	1
$\text{Fst}(\Pi^{\text{gn}}\alpha:\sigma_1.\sigma_2)$	$\stackrel{\text{def}}{=}$	1
$\text{Fst}(\Pi^{\text{ap}}\alpha:\sigma_1.\sigma_2)$	$\stackrel{\text{def}}{=}$	$\Pi\alpha:\text{Fst}(\sigma_1).\text{Fst}(\sigma_2)$
$\text{Fst}(\Sigma\alpha:\sigma_1.\sigma_2)$	$\stackrel{\text{def}}{=}$	$\Sigma\alpha:\text{Fst}(\sigma_1).\text{Fst}(\sigma_2)$

$\Gamma \vdash \text{Fst}(M) \gg c$

$\frac{\alpha/m \in \text{Dom}(\Gamma)}{\Gamma \vdash \text{Fst}(m) \gg \alpha}$	$\frac{}{\Gamma \vdash \text{Fst}(\star) \gg \star}$	$\frac{}{\Gamma \vdash \text{Fst}(\langle c \rangle) \gg c}$
$\frac{}{\Gamma \vdash \text{Fst}(\langle e \rangle) \gg \star}$	$\frac{}{\Gamma \vdash \text{Fst}(\lambda^{\text{gn}}\alpha/m:\sigma.M) \gg \star}$	
$\frac{\Gamma, \alpha/m:\sigma \vdash \text{Fst}(M) \gg c}{\Gamma \vdash \text{Fst}(\lambda^{\text{ap}}\alpha/m:\sigma.M) \gg \lambda\alpha:\text{Fst}(\sigma).c}$	$\frac{\Gamma \vdash \text{Fst}(M_1) \gg c_1 \quad \Gamma \vdash \text{Fst}(M_2) \gg c_2}{\Gamma \vdash \text{Fst}(M_1 \cdot M_2) \gg c_1 c_2}$	
	$\frac{}{\Gamma \vdash \text{Fst}(\langle M_1, M_2 \rangle) \gg \langle c_1, c_2 \rangle}$	
$\frac{\Gamma \vdash \text{Fst}(M) \gg c}{\Gamma \vdash \text{Fst}(\pi_i M) \gg \pi_i c}$	$\frac{\Gamma \vdash \text{Fst}(M) \gg c}{\Gamma \vdash \text{Fst}(\text{let } x = e \text{ in } M) \gg c}$	

Fig. 2. Static portions

into static and dynamic components. The main difference (apart from the extra features we support here), is that their “non-standard” rules determined not only a static portion, but also a dynamic portion, even though only the static portion ended up being relevant to typing judgements.

Signatures. Signatures forms include signatures for atomic modules, generative and application functors, and pairs. The signatures for functors and for pairs are dependent, so one might expect to write them with a twinned left-hand-side, like $\Sigma\alpha/m:\sigma_1.\sigma_2$. However, since a module can never appear within a signature (signatures being static phrases), the m variable will never be used, so we don’t bother to write it. (As usual, when α does not appear free in σ_2 , we write $\Sigma\alpha:\sigma_1.\sigma_2$ as $\sigma_1 \times \sigma_2$.)

In a twinned binding $\alpha/m:\sigma$, or in the binding $\alpha:\sigma$ within a dependent signature, α stands for the static portion of some module belonging to σ . Thus, α will have kind $\text{Fst}(\sigma)$, which stands for the static portion of σ . The definition of Fst appears in Figure 2. Whenever $m : \sigma$ and $\text{Fst}(m) \gg c$, we have $c : \text{Fst}(\sigma)$.

Higher-order singletons. The primitive singleton kind $S(c)$ is well-formed only when c is a type. However, singletons at higher kinds (written $S(c : k)$) are definable using dependent kinds. For example, $S(c : k_1 \rightarrow k_2) = \Pi\alpha:k_1.S(c \alpha : k_2)$, since a constructor is equivalent to c in kind $k_1 \rightarrow k_2$ precisely when it takes an argument of kind k_1 and does with it whatever c does with it.

We can use the same technique to define singleton signatures. Suppose $c : \text{Fst}(\sigma)$. Then the singleton signature $S(c : \sigma)$ contains all pure modules in σ whose static component is equivalent to c . For instance $S(c : \langle k \rangle) = \langle S(c : k) \rangle$. The definitions of higher-order singletons are given in Figure 3. Note that when the static component of σ is trivial (i.e., when $\text{Fst}(\sigma) = 1$), $S(c : \sigma) = \sigma$, since all static components are equivalent at kind 1.

$S(c : k)$	$:$	$kind$
$S(c : 1)$	$\stackrel{\text{def}}{=}$	1
$S(c : \top)$	$\stackrel{\text{def}}{=}$	$S(c)$
$S(c : S(c'))$	$\stackrel{\text{def}}{=}$	$S(c)$
$S(c : \Pi\alpha:k_1.k_2)$	$\stackrel{\text{def}}{=}$	$\Pi\alpha:k_1.S(c\ \alpha : k_2)$
$S(c : \Sigma\alpha:k_1.k_2)$	$\stackrel{\text{def}}{=}$	$S(\pi_1 c : k_1) \times S(\pi_2 c : [\pi_1 c / \alpha]k_2)$
$S(c : \sigma)$	$:$	$signature$
$S(c : 1)$	$\stackrel{\text{def}}{=}$	1
$S(c : \langle k \rangle)$	$\stackrel{\text{def}}{=}$	$\langle S(c : k) \rangle$
$S(c : \langle \tau \rangle)$	$\stackrel{\text{def}}{=}$	$\langle \tau \rangle$
$S(c : \Pi^{\text{gn}}\alpha:\sigma_1.\sigma_2)$	$\stackrel{\text{def}}{=}$	$\Pi^{\text{gn}}\alpha:\sigma_1.\sigma_2$
$S(c : \Pi^{\text{ap}}\alpha:\sigma_1.\sigma_2)$	$\stackrel{\text{def}}{=}$	$\Pi^{\text{ap}}\alpha:\sigma_1.S(c\ \alpha : \sigma_2)$
$S(c : \Sigma\alpha:\sigma_1.\sigma_2)$	$\stackrel{\text{def}}{=}$	$S(\pi_1 c : \sigma_1) \times S(\pi_2 c : [\pi_1 c / \alpha]\sigma_2)$

Fig. 3. Higher-order singletons

$\vdash \Gamma \text{ ok}$	context formation	$\Gamma \vdash \sigma : \text{sig}$	signature formation
$\Gamma \vdash k : \text{kind}$	kind formation	$\Gamma \vdash \sigma \equiv \sigma' : \text{sig}$	signature equivalence
$\Gamma \vdash k \equiv k'$	kind equivalence	$\Gamma \vdash \sigma \leq \sigma'$	subsignature
$\Gamma \vdash k \leq k'$	subkind	$\Gamma \vdash e : \tau$	term formation
$\Gamma \vdash c : k$	constructor formation	$\Gamma \vdash_{\kappa} M : \sigma$	module formation
$\Gamma \vdash c \equiv c' : k$	constructor equivalence	$\Gamma \vdash \text{Fst}(M) \gg c$	static portion

Fig. 4. Static semantic judgements

Semantics. The static semantics is given by several judgements summarized in Figure 4. The rules are exactly those from Crary [2017], with the sole exception that the typing rule for polymorphic functions is replaced by the one given below, which includes a value restriction.

The top-level judgement is the typing judgement for modules, written $\Gamma \vdash_{\kappa} M : \sigma$, where the *purity class* κ is either P, indicating that the module is pure (unsealed), or I, indicating that it is impure (sealed). A “forget” rule allows pure modules to be viewed as impure.

The dynamic semantics is given by a standard, call-by-value, structured operational semantics, written $\Gamma \vdash e \mapsto e'$ and $\Gamma \vdash M \mapsto M'$. (It is convenient to be able to evaluate open terms, but in the typical case in which Γ is empty, we will omit the turnstile.) The value forms are:

$$\begin{aligned}
 v &::= x \mid \star \mid \lambda x:\tau.e \mid \langle v, v \rangle \mid \Lambda\alpha:k.e \mid \text{pack}[c, v] \text{ as } \exists\alpha:k.\tau \\
 V &::= m \mid \star \mid \langle c \rangle \mid \langle v \rangle \mid \langle V, V \rangle \mid \lambda^{\text{gn}}\alpha/m:\sigma.M \mid \lambda^{\text{ap}}\alpha/m:\sigma.M
 \end{aligned}$$

We write $e \downarrow$ or $M \downarrow$ to mean that e or M evaluates to a value.

Two rules illustrate issues that arise in the dynamic semantics. The evaluation rule for sealed modules immediately removes the seal:

$$\overline{\Gamma \vdash (M :> \sigma) \mapsto M}$$

In essence, this step is the computational effect that the type system tracks. Rules that substitute for module variables, such as the beta rules for functors, also must substitute for the twinned constructor variable, which is obtained using the Fst judgement:

$$\frac{\Gamma \vdash \text{Fst}(V_2) \gg c_2}{\Gamma \vdash (\lambda^{\text{gn}} \alpha / m : \sigma . M_1) V_2 \mapsto [c_2, V_2 / \alpha, m] M_1}$$

This rule works because well-formed module values are always pure, so their static portion can always be obtained.

The value restriction. In the typing rule for polymorphic functions, we require that the body be a static value:

$$\frac{\Gamma \vdash k : \text{kind} \quad \Gamma, \alpha : k \vdash sv : \tau}{\Gamma \vdash \Lambda \alpha : k . sv : \forall \alpha : k . \tau}$$

where a static value (sv) is like an ordinary value (v) except variables are excluded:

$$sv ::= \star \mid \lambda x : \tau . e \mid \langle sv, sv \rangle \mid \Lambda \alpha : k . sv \mid \text{pack } [c, sv] \text{ as } \exists \alpha : k . \tau$$

We distinguish between static and ordinary values and use the former because we wish for well-formedness to be preserved by the substitution of any appropriately typed term—even a non-value—for a variable, and that is not the case with ordinary values. This is necessary to have a useful contextual-equivalence functionality lemma.

The inclusion of the value restriction has no significant effect on the development of logical equivalence given in Cray [2017], but it is essential for the phase separation translation to be fully abstract.

Metatheory. We can now state some metatheoretic results from Cray [2017]:

LEMMA 2.1.

- If $\Gamma \vdash_P M : \sigma$ then $\Gamma \vdash \text{Fst}(M) : \text{Fst}(\sigma)$.
- If $\vdash \Gamma \text{ ok}$ and $\Gamma \vdash c : k$ then $\Gamma \vdash c : S(c : k)$ and $\Gamma, \alpha : S(c : k) \vdash \alpha \equiv c : k$.
- If $\vdash \Gamma \text{ ok}$ and $\Gamma \vdash_P M : \sigma$ then $\Gamma \vdash_P M : S(\text{Fst}(M) : \sigma)$.
- If $\Gamma \vdash_I V : \sigma$ then $\Gamma \vdash_P V : \sigma$.

THEOREM 2.2 (TYPE PRESERVATION). If $\vdash \Gamma \text{ ok}$ then:

- If $\Gamma \vdash e : \tau$ and $\Gamma \vdash e \mapsto e'$ then $\Gamma \vdash e' : \tau$.
- If $\Gamma \vdash_K M : \sigma$ and $\Gamma \vdash M \mapsto M'$ then $\Gamma \vdash_K M' : \sigma$.
- If $\Gamma \vdash_P M : \sigma$ and $\Gamma \vdash M \mapsto M'$ then $\Gamma \vdash \text{Fst}(M) \equiv \text{Fst}(M') : \text{Fst}(\sigma)$.

THEOREM 2.3 (PROGRESS). If $\vdash e : \tau$ then either e is a value or takes a step. If $\vdash_K M : \sigma$ then either M is a value or takes a step.

THEOREM 2.4 (DETERMINISM).

- If $\Gamma \vdash e \mapsto e_1$ and $\Gamma \vdash e \mapsto e_2$ then $e_1 = e_2$.
- If $\Gamma \vdash M \mapsto M_1$ and $\Gamma \vdash M \mapsto M_2$ then $M_1 = M_2$.
- If $\Gamma \vdash \text{Fst}(M) \gg c_1$ and $\Gamma \vdash \text{Fst}(M) \gg c_2$ then $c_1 = c_2$.

3 PHASE SEPARATION

The target language for our translation is the subset of the source language obtained by deleting all the module-oriented forms. Note that the syntactic classes of kinds and constructors contain no module-oriented forms, and are thus unchanged. (This is an advantage of Dreyer’s device [2005] wherein Fst is a judgement, rather than a syntactic form.)

Using a fragment of the source as the target dramatically simplifies several aspects of the development: (1) We need not develop the metatheory of the target language; the source's metatheory applies directly. (2) The type translation is the identity. (3) Full abstraction is a direct consequence of the dynamic correctness theorem, since every target observation is a source observation, and every source observation can be translated to a target observation.

We express phase separation as a type-directed translation [Harper and Morrisett 1995] using three judgements, one each for terms, pure modules, and impure modules. The term translation judgement is $\Gamma \vdash e : \tau \rightsquigarrow \bar{e}$, meaning that $\Gamma \vdash e : \tau$ and $\bar{e}$ is a translation of e . Since the type translation is the identity, $\bar{e}$ will have the same type as e .

Different translation derivations can ostensibly result in different translations, but we can show that the translation is coherent [Breazu-Tannen et al. 1991], meaning that all of a term's translations are equivalent. Coherence is another simple corollary of dynamic correctness.

The term translation does very little, since most term forms do not pertain to modules. Its rules are given in Appendix A. The main judgements are the phase-separation judgements for modules. For pure modules, the judgement is written:

$$\Gamma \vdash_p M : \sigma \rightsquigarrow [c, e]$$

Here, M is separated into its static portion c and its dynamic portion e . Recall that the module calculus has its own internal concept of static components ($\text{Fst}(M)$). These notions agree: if also $\Gamma \vdash \text{Fst}(M) \gg c'$ then $c = c'$.

The types of c and e are governed by the phase separation judgement for signatures:

$$\sigma \rightsquigarrow [\alpha:k. \tau]$$

Here σ is separated into its static portion k and its dynamic portion τ (in which τ can refer to α , which stands for the actual static component). Thus, in the above, c belongs to kind k , and e belongs to type $[c/\alpha]\tau$.

The best way to understand the translation is through the phase-separation rules for signatures. A static atom is all static, so $\langle k \rangle \rightsquigarrow [_ : k. \text{unit}]$. Conversely, a dynamic atom is all dynamic, so $\langle \tau \rangle \rightsquigarrow [_ : 1. \tau]$. A trivial module is trivial in both components, so $1 \rightsquigarrow [_ : 1. \text{unit}]$.

In an applicative functor, the static portion maps the static portion of the argument to the static portion of the result, while the dynamic portion maps both portions of the argument to the dynamic portion of the result:

$$\frac{\sigma_1 \rightsquigarrow [\alpha_1:k_1. \tau_1] \quad \sigma_2 \rightsquigarrow [\alpha_2:k_2. \tau_2]}{\Pi^{\text{ap}} \alpha:\sigma_1.\sigma_2 \rightsquigarrow [\beta : \Pi \alpha:k_1.k_2. \forall \alpha:k_1. [\alpha/\alpha_1]\tau_1 \rightarrow [\beta \alpha/\alpha_2]\tau_2]}$$

In a pair, the static (dynamic) portion is the pair of the static (dynamic) portions of the constituents:

$$\frac{\sigma_1 \rightsquigarrow [\alpha_1:k_1. \tau_1] \quad \sigma_2 \rightsquigarrow [\alpha_2:k_2. \tau_2]}{\Sigma \alpha:\sigma_1.\sigma_2 \rightsquigarrow [\beta : \Sigma \alpha:k_1.k_2. [\pi_1 \beta / \alpha_1]\tau_1 \times [\pi_1 \beta, \pi_2 \beta / \alpha, \alpha_2]\tau_2]}$$

These rules are essentially the rules from Harper, *et al.* [1990] (adapted for dependent kinds), but they did not account for generative functors. For generative functors, we adapt the translation of Shan [2006] and Rossberg, *et al.* [2014].

The tricky point is how to phase-separate impure (*i.e.*, sealed) modules. Our translation wishes to separate modules into their static and dynamic components, but sealed modules are not supposed to have a static component, since their abstract type components are notionally determined at run-time.⁶ Any translation that makes them available statically will not preserve abstraction.

⁶Indeed, when first-class modules are added, they are *actually* determined at run-time.

$$\boxed{\sigma \rightsquigarrow [\alpha:k. \tau]}$$

$$\begin{array}{c} \overline{1 \rightsquigarrow [_ : 1. \text{unit}]} \quad \overline{(\llbracket k \rrbracket) \rightsquigarrow [_ : k. \text{unit}]} \quad \overline{\langle \tau \rangle \rightsquigarrow [_ : 1. \tau]} \\[10pt] \frac{\sigma_1 \rightsquigarrow [\alpha_1:k_1. \tau_1] \quad \sigma_2 \rightsquigarrow [\alpha_2:k_2. \tau_2]}{\Pi^{\text{gn}} \alpha:\sigma_1.\sigma_2 \rightsquigarrow [_ : 1. \forall \alpha:k_1. [\alpha/\alpha_1]\tau_1 \rightarrow \exists \alpha_2:k_2.\tau_2]} \\[10pt] \frac{\sigma_1 \rightsquigarrow [\alpha_1:k_1. \tau_1] \quad \sigma_2 \rightsquigarrow [\alpha_2:k_2. \tau_2]}{\Pi^{\text{ap}} \alpha:\sigma_1.\sigma_2 \rightsquigarrow [\beta : \Pi \alpha:k_1.k_2. \forall \alpha:k_1. [\alpha/\alpha_1]\tau_1 \rightarrow [\beta \alpha/\alpha_2]\tau_2]} \\[10pt] \frac{\sigma_1 \rightsquigarrow [\alpha_1:k_1. \tau_1] \quad \sigma_2 \rightsquigarrow [\alpha_2:k_2. \tau_2]}{\Sigma \alpha:\sigma_1.\sigma_2 \rightsquigarrow [\beta : \Sigma \alpha:k_1.k_2. [\pi_1 \beta/\alpha_1]\tau_1 \times [\pi_2 \beta/\alpha_2]\tau_2]} \end{array}$$

Fig. 5. The signature translation

But once a sealed module has executed, its result value *will* be pure (recall part 4 of Lemma 2.1), and therefore it will have static and dynamic components. Thus, the appropriate translation of an impure module is a term that dynamically computes the static and dynamic portions of that eventual value. Thus we write:

$$\Gamma \vdash_1 M : \sigma \rightsquigarrow e$$

where e belongs to $\exists \alpha:k.\tau$ (assuming $\sigma \rightsquigarrow [\alpha:k. \tau]$).

Now we can give the rule for a generative functor, which returns an impure module. Its static portion is trivial, while its dynamic portion maps both portions of the argument to an existential package containing the static and dynamic portions of the module value that the functor eventually returns:

$$\frac{\sigma_1 \rightsquigarrow [\alpha_1:k_1. \tau_1] \quad \sigma_2 \rightsquigarrow [\alpha_2:k_2. \tau_2]}{\Pi^{\text{gn}} \alpha:\sigma_1.\sigma_2 \rightsquigarrow [_ : 1. \forall \alpha:k_1. [\alpha/\alpha_1]\tau_1 \rightarrow \exists \alpha_2:k_2.\tau_2]}$$

For convenience, the signature translation rules are collected in Figure 5. The module translation is given in Figures 6, and 7. Most of its rules are dictated by the signature phase-separation rules given above. We touch on a few points that are interesting or subtle:

The translation's calling convention is given by the context translation $\bar{\Gamma}$, which is formalized in Figure 8. Constructor and term bindings are translated directly, as one would expect. In a module binding α/m , we introduce two variables—a constructor variable α and a term variable $\hat{m}$ —to represent the module's static and dynamic portions. Here we assume that there are infinitely many term variables, so for every module variable m we can associate a term variable $\hat{m}$ that is not otherwise used.

With the calling convention established, we can consider the translation rules for functors. For applicative functors, the rule states:

$$\frac{\Gamma \vdash \sigma_1 : \text{sig} \quad \sigma_1 \rightsquigarrow [\alpha:k_1. \tau_1] \quad \Gamma, \alpha/m:\sigma_1 \vdash_1 M : \sigma_2 \rightsquigarrow [c, e]}{\Gamma \vdash_1 \lambda^{\text{ap}} \alpha/m:\sigma_1.M : \Pi^{\text{ap}} \alpha:\sigma_1.\sigma_2 \rightsquigarrow [\lambda \alpha:k_1.c, \Lambda \alpha:k_1.\lambda \hat{m}:\tau_1.e]}$$

The functor's static portion $\lambda \alpha:k_1.c$ takes the static portion of the functor argument ($\alpha:k_1$) to the static portion of the body (c). In accordance with the phase distinction, there is no dependence on

$$\boxed{\Gamma \vdash_p M : \sigma \rightsquigarrow [c, e]}$$

$$\begin{array}{c}
\frac{(\alpha/m : \sigma) \in \Gamma}{\Gamma \vdash_p m : S(\alpha : \sigma) \rightsquigarrow [\alpha, \hat{m}]} \quad \frac{}{\Gamma \vdash \star : 1 \rightsquigarrow [\star, \star]} \\
\\
\frac{\Gamma \vdash c : k}{\Gamma \vdash \langle c \rangle : \langle k \rangle \rightsquigarrow [c, \star]} \quad \frac{\Gamma \vdash e : \tau \rightsquigarrow \bar{e}}{\Gamma \vdash \langle e \rangle : \langle \tau \rangle \rightsquigarrow [\star, \bar{e}]} \\
\\
\frac{\Gamma \vdash \sigma_1 : \text{sig} \quad \sigma_1 \rightsquigarrow [\alpha : k_1. \tau_1] \quad \Gamma, \alpha/m : \sigma_1 \vdash_1 M : \sigma_2 \rightsquigarrow e}{\Gamma \vdash_p \lambda^{\text{gn}} \alpha / m : \sigma_1. M : \Pi^{\text{gn}} \alpha : \sigma_1. \sigma_2 \rightsquigarrow [\star, \Lambda \alpha : k_1. \lambda \hat{m} : \tau_1. e]} \\
\\
\frac{\Gamma \vdash \sigma_1 : \text{sig} \quad \sigma_1 \rightsquigarrow [\alpha : k_1. \tau_1] \quad \Gamma, \alpha/m : \sigma_1 \vdash_p M : \sigma_2 \rightsquigarrow [c, e]}{\Gamma \vdash_p \lambda^{\text{ap}} \alpha / m : \sigma_1. M : \Pi^{\text{ap}} \alpha : \sigma_1. \sigma_2 \rightsquigarrow [\lambda \alpha : k_1. c, \Lambda \alpha : k_1. \lambda \hat{m} : \tau_1. e]} \\
\\
\frac{\Gamma \vdash_p M_1 : \Pi^{\text{ap}} \alpha : \sigma_1. \sigma_2 \rightsquigarrow [c_1, e_1] \quad \Gamma \vdash_p M_2 : \sigma_1 \rightsquigarrow [c_2, e_2]}{\Gamma \vdash_p M_1 \cdot M_2 : [c_2/\alpha] \sigma_2 \rightsquigarrow [c_1 c_2, e_1 [c_2] e_2]} \\
\\
\frac{\Gamma \vdash_p M_1 : \sigma_1 \rightsquigarrow [c_1, e_1] \quad \Gamma \vdash_p M_2 : \sigma_2 \rightsquigarrow [c_2, e_2]}{\Gamma \vdash_p \langle M_1, M_2 \rangle : \sigma_1 \times \sigma_2 \rightsquigarrow [\langle c_1, c_2 \rangle, \langle e_1, e_2 \rangle]} \\
\\
\frac{\Gamma \vdash_p M : \Sigma \alpha : \sigma_1. \sigma_2 \rightsquigarrow [c, e]}{\Gamma \vdash_p \pi_1 M : \sigma_1 \rightsquigarrow [\pi_1 c, \pi_1 e]} \quad \frac{\Gamma \vdash_p M : \Sigma \alpha : \sigma_1. \sigma_2 \rightsquigarrow [c, e]}{\Gamma \vdash_p \pi_2 M : [\pi_1 c/\alpha] \sigma_2 \rightsquigarrow [\pi_2 c, \pi_2 e]} \\
\\
\frac{\Gamma \vdash_p e : \tau \rightsquigarrow e_1 \quad \Gamma, x : \tau \vdash_p M : \sigma \rightsquigarrow [c, e_2]}{\Gamma \vdash_p \text{let } x = e \text{ in } M : \sigma \rightsquigarrow [c, \text{let } x = e_1 \text{ in } e_2]} \quad \frac{\Gamma \vdash_p M : \sigma_1 \rightsquigarrow [c, e] \quad \Gamma \vdash \sigma_1 \leq \sigma_2 \rightsquigarrow f}{\Gamma \vdash_p M : \sigma_2 \rightsquigarrow [c, f [c] e]}
\end{array}$$

Fig. 6. The pure module translation

the dynamic portion of the argument. The functor's dynamic portion takes the static and dynamic portions of the argument ($\alpha : k_1$ and $\hat{m} : \tau_1$)—thereby following the calling convention for $\alpha/m : \sigma_1$ —to the dynamic portion of the body (e).

The applicative functor application rule states (in the pure case):

$$\frac{\Gamma \vdash_p M_1 : \Pi^{\text{ap}} \alpha : \sigma_1. \sigma_2 \rightsquigarrow [c_1, e_1] \quad \Gamma \vdash_p M_2 : \sigma_1 \rightsquigarrow [c_2, e_2]}{\Gamma \vdash_p M_1 \cdot M_2 : [c_2/\alpha] \sigma_2 \rightsquigarrow [c_1 c_2, e_1 [c_2] e_2]}$$

This simply applies the functor's static portion to the argument's, and applies the functor's dynamic portion to the argument's static and dynamic portion. The impure case does the same thing, but first unpacks the functor from an existential package, and afterwards repacks the result.

Observe that, unless one were to do something obviously foolish, both of these rules are completely determined by the signature translation. The same is true for the translation of generative functors:

$$\frac{\Gamma \vdash \sigma_1 : \text{sig} \quad \sigma_1 \rightsquigarrow [\alpha : k_1. \tau_1] \quad \Gamma, \alpha/m : \sigma_1 \vdash_1 M : \sigma_2 \rightsquigarrow e}{\Gamma \vdash_p \lambda^{\text{gn}} \alpha / m : \sigma_1. M : \Pi^{\text{gn}} \alpha : \sigma_1. \sigma_2 \rightsquigarrow [\star, \Lambda \alpha : k_1. \lambda \hat{m} : \tau_1. e]}$$

$$\boxed{\Gamma \vdash_1 M : \sigma \rightsquigarrow e}$$

$$\frac{\Gamma \vdash_1 M_1 : \Pi^{\text{gn}} \alpha : \sigma_1 . \sigma_2 \rightsquigarrow e_1 \quad \Gamma \vdash_P M_2 : \sigma_1 \rightsquigarrow [c_2, e_2]}{\Gamma \vdash_1 M_1 M_2 : [c_2/\alpha] \sigma_2 \rightsquigarrow \text{unpack } [_, f] = e_1 \text{ in } f [c_2] e_2}$$

$$\frac{\Gamma \vdash_1 M_1 : \Pi^{\text{ap}} \alpha : \sigma_1 . \sigma_2 \rightsquigarrow e_1 \quad \Gamma \vdash_P M_2 : \sigma_1 \rightsquigarrow [c_2, e_2] \quad \sigma_2 \rightsquigarrow [\beta : k_2 . \tau_2]}{\Gamma \vdash_1 M_1 \cdot M_2 : [c_2/\alpha] \sigma_2 \rightsquigarrow \text{unpack } [\gamma, f] = e_1 \text{ in pack } [\gamma \ c_2, f [c_2] e_2] \text{ as } [c_2/\alpha] (\exists \beta : k_2 . \tau_2)}$$

$$\frac{\Gamma \vdash M_1 : \sigma_1 \rightsquigarrow e_1 \quad \Gamma \vdash M_2 : \sigma_2 \rightsquigarrow e_2 \quad \sigma_1 \rightsquigarrow [\alpha_1 : k_1 . \tau_1] \quad \sigma_2 \rightsquigarrow [\alpha_2 : k_2 . \tau_2]}{\Gamma \vdash \langle M_1, M_2 \rangle : \sigma_1 \times \sigma_2 \rightsquigarrow \text{unpack } [\alpha_1, x_1] = e_1 \text{ in unpack } [\alpha_2, x_2] = e_2 \text{ in pack } [\langle \alpha_1, \alpha_2 \rangle, \langle x_1, x_2 \rangle] \text{ as } \exists \beta : (k_1 \times k_2) . [\pi_1 \beta / \alpha_1] \tau_1 \times [\pi_2 \beta / \alpha_2] \tau_2}$$

$$\frac{\Gamma \vdash e : \exists \alpha : k . \tau \rightsquigarrow e_1 \quad \Gamma, \alpha : k, x : \tau \vdash_1 M : \sigma \rightsquigarrow e_2 \quad \Gamma \vdash \sigma : \text{sig}}{\Gamma \vdash_1 \text{unpack } [\alpha, x] = e \text{ in } (M : \sigma) : \sigma \rightsquigarrow \text{unpack } [\alpha, x] = e_1 \text{ in } e_2}$$

$$\frac{\Gamma \vdash e : \tau \rightsquigarrow e_1 \quad \Gamma, x : \tau \vdash_1 M : \sigma \rightsquigarrow e_2}{\Gamma \vdash \text{let } x = e \text{ in } M : \sigma \rightsquigarrow \text{let } x = e_1 \text{ in } e_2}$$

$$\frac{\Gamma \vdash M_1 : \sigma_1 \rightsquigarrow e_1 \quad \Gamma, \alpha / m : \sigma_2 \vdash M_2 : \sigma_2 \rightsquigarrow e_2 \quad \Gamma \vdash \sigma_2 : \text{sig}}{\Gamma \vdash \text{let } \alpha / m = M_1 \text{ in } (M_2 : \sigma_2) : \sigma_2 \rightsquigarrow \text{unpack } [\alpha, \hat{m}] = e_1 \text{ in } e_2}$$

$$\frac{\Gamma \vdash_1 M : \sigma \rightsquigarrow e \quad \Gamma \vdash_P M : \sigma \rightsquigarrow [c, e] \quad \sigma \rightsquigarrow [\alpha : k . \tau]}{\Gamma \vdash_1 (M :> \sigma) : \sigma \rightsquigarrow e \quad \Gamma \vdash_1 M : \sigma \rightsquigarrow \text{pack } [c, e] \text{ as } \exists \alpha : k . \tau}$$

$$\frac{\Gamma \vdash_1 M : \sigma_1 \rightsquigarrow e \quad \Gamma \vdash \sigma_1 \leq \sigma_2 \rightsquigarrow f \quad \sigma_2 \rightsquigarrow [\alpha : k_2 . \tau_2]}{\Gamma \vdash_1 M : \sigma_2 \rightsquigarrow \text{unpack } [\alpha, x] = e \text{ in pack } [\alpha, f [\alpha] x] \text{ as } \exists \alpha : k_2 . \tau_2}$$

Fig. 7. The impure module translation

$$\begin{array}{lcl}
\bar{\epsilon} & \stackrel{\text{def}}{=} & \epsilon \\
\overline{\Gamma, \alpha : k} & \stackrel{\text{def}}{=} & \bar{\Gamma}, \alpha : k \\
\overline{\Gamma, x : \tau} & \stackrel{\text{def}}{=} & \bar{\Gamma}, x : \tau \\
\overline{\Gamma, \alpha / m : \sigma} & \stackrel{\text{def}}{=} & \bar{\Gamma}, \alpha : k, \hat{m} : \tau \quad \text{where } \sigma \rightsquigarrow [\alpha : k . \tau]
\end{array}$$

Fig. 8. The context translation

The static portion of a generative functor is trivial. The dynamic portion takes in the static and dynamic portions of the argument (again $\alpha : k_1$ and $\hat{m} : \tau_1$)—again following the calling convention—and then computes an existential package (e) containing the static and dynamic portions of the result.

The generative functor application rule:

$$\frac{\Gamma \vdash_1 M_1 : \Pi^{\text{gn}} \alpha : \sigma_1. \sigma_2 \rightsquigarrow e_1 \quad \Gamma \vdash_P M_2 : \sigma_1 \rightsquigarrow [c_2, e_2]}{\Gamma \vdash_1 M_1 M_2 : [c_2/\alpha] \sigma_2 \rightsquigarrow \text{unpack}[_, f] = e_1 \text{ in } f[c_2] e_2}$$

after extracting the functor's dynamic portion from a trivial existential package simply applies it to the argument's static and dynamic portion.

Again, these rules are determined by the signature translation. The same is true of the rules governing atomic modules and pairs. The translation of let binding is determined simply by the calling convention:

$$\frac{\Gamma \vdash M_1 : \sigma_1 \rightsquigarrow e_1 \quad \Gamma, \alpha/m : \sigma_2 \vdash M_2 : \sigma_2 \rightsquigarrow e_2 \quad \Gamma \vdash \sigma_2 : \text{sig}}{\Gamma \vdash \text{let } \alpha/m = M_1 \text{ in } (M_2 : \sigma_2) : \sigma_2 \rightsquigarrow \text{unpack}[\alpha, \hat{m}] = e_1 \text{ in } e_2}$$

Since M_1 is impure, its translation is an existential package that must be unpacked to follow the calling convention. Once this is done, the right-hand-side is just passed through. Since σ_2 does not mention α , the type of e_2 also will not mention α .

The translation rule corresponding to the “forgetting” rule takes the statically determined static and dynamic components and rolls them into an existential package:

$$\frac{\Gamma \vdash_P M : \sigma \rightsquigarrow [c, e] \quad \sigma \rightsquigarrow [\alpha:k. \tau]}{\Gamma \vdash_1 M : \sigma \rightsquigarrow \text{pack}[c, e] \text{ as } \exists \alpha:k. \tau}$$

Interestingly, this leaves nothing for the sealing rule to do:

$$\frac{\Gamma \vdash_1 M : \sigma \rightsquigarrow e}{\Gamma \vdash_1 (M :> \sigma) : \sigma \rightsquigarrow e}$$

All the sealing rule does is require that M be translated impurely. This induces the forgetting rule to do the work of sealing M up as an existential package.

Retyping principles. Two technically tricky points pertain to the module calculus's retyping principles. The first such principle is the subsumption rule:

$$\frac{\Gamma \vdash_K M : \sigma_1 \quad \Gamma \vdash \sigma_1 \leq \sigma_2}{\Gamma \vdash_K M : \sigma_2}$$

If $\sigma_1 \leq \sigma_2$ and $\sigma_i \rightsquigarrow [\alpha:k_i. \tau_i]$, we can show that $k_1 \leq k_2$. If we had subtyping (and the full $F_{\leq}$ subtyping rule [Curien and Ghelli 1992]) we could also show that $\tau_1 \leq \tau_2$. But we do not have subtyping, and it is not the case that $\tau_1 \equiv \tau_2$, since $k' \leq k$ does not imply $\forall \alpha:k. \tau \equiv \forall \alpha:k'. \tau$. Consequently, the subsumption rule must apply a $\tau_1 \rightarrow \tau_2$ coercion to the dynamic portion:

$$\frac{\frac{\Gamma \vdash_P M : \sigma_1 \rightsquigarrow [c, e] \quad \Gamma \vdash \sigma_1 \leq \sigma_2 \rightsquigarrow f}{\Gamma \vdash_P M : \sigma_2 \rightsquigarrow [c, f[c] e]}}{\Gamma \vdash_1 M : \sigma_1 \rightsquigarrow e \quad \Gamma \vdash \sigma_1 \leq \sigma_2 \rightsquigarrow f \quad \sigma_2 \rightsquigarrow [\alpha:k_2. \tau_2]}{\Gamma \vdash_1 M : \sigma_2 \rightsquigarrow \text{unpack}[\alpha, x] = e \text{ in } \text{pack}[\alpha, f[\alpha]x] \text{ as } \exists \alpha:k_2. \tau_2}$$

The coercion is computed by a judgement $\Gamma \vdash \sigma_1 \leq \sigma_2 \rightsquigarrow f$. It is essentially a large eta-expansion; it unrolls a term representing the dynamic portion of a module belonging to σ_1 , and then re-rolls it at the type corresponding to σ_2 . Details appear in Appendix A. In a practical implementation, this coercion can very frequently be inlined. Moreover, for fragments without higher-order functors (such as Standard ML), the coercion is always the identity.

The second retyping principle is the calculus's extensional typing rules, such as this rule for module pairs:

$$\frac{\Gamma \vdash_P \pi_1 M : \sigma_1 \quad \Gamma \vdash_P \pi_2 M : \sigma_2}{\Gamma \vdash_P M : \sigma_1 \times \sigma_2}$$

The extensional typing rules say that a module belongs to a signature if uses of M produce appropriate results. For instance, M belongs to $\sigma_1 \times \sigma_2$ if its left projection belongs to σ_1 and its right to σ_2 . These rules are never needed when M is an introduction form, but they allow one to ascribe stronger signatures to paths (*i.e.*, series of eliminations starting from a variable) than the elimination rules permit [Harper and Lillibridge 1994]. For example, they allow one to give a path a translucent signature that refers to itself, a procedure whimsically called “selfification.” Extensional typing is necessary for higher-order singleton kinds and signatures to work properly (recall parts 2 and 3 of Lemma 2.1).

However, the extensional typing rules cause a problem for the translation. Suppose M translated to $[c, e]$ before using extensionality. The translation rule corresponding to the above rule would produce $[\langle \pi_1 c, \pi_2 c \rangle, \langle \pi_1 e, \pi_2 e \rangle]$. A similar thing would happen with the extensional typing rule for functors.

Unfortunately, this eta-expansion is not only inefficient, it is incorrect. In an eager operational semantics such as ours, e and $\langle \pi_1 e, \pi_2 e \rangle$ have observably different behavior whenever e has effects.

The solution to this problem is to exploit the fact that the extensional typing rules are needed only for paths. Our translation omits the extensional typing rules, and replaces them with a single “selfifying” translation rule for variables:

$$\frac{(\alpha/m : \sigma) \in \Gamma}{\Gamma \vdash_P m : S(\alpha : \sigma) \rightsquigarrow [\alpha, \hat{m}]}$$

Note the typing rule for variables gives m the signature σ , but the translation here gives it $S(\alpha : \sigma)$. Because of this discrepancy, the translation rules are not in perfect correspondence with the typing rules, but we can nevertheless show that all well-typed expressions translate and vice versa:

THEOREM 3.1. *If $\vdash \Gamma$ ok then:*

- $\Gamma \vdash \sigma_1 \leq \sigma_2$ iff $\Gamma \vdash \sigma_1 \leq \sigma_2 \rightsquigarrow f$ (for some f).
- $\Gamma \vdash_P M : \sigma$ iff $\Gamma \vdash_P M : \sigma \rightsquigarrow [c, e]$ (for some c, e).
- $\Gamma \vdash_1 M : \sigma$ iff $\Gamma \vdash_1 M : \sigma \rightsquigarrow e$ (for some e).
- $\Gamma \vdash e : \tau$ iff $\Gamma \vdash e : \tau \rightsquigarrow \bar{e}$ (for some $\bar{e}$).

We will need more machinery before we can show the translation’s full correctness, but at this point we can show its static correctness:

LEMMA 3.2 (STATIC CORRECTNESS). *If $\vdash \Gamma$ ok then:*

- If $\Gamma \vdash \sigma : \text{sig}$ and $\sigma \rightsquigarrow [\alpha : k, \tau]$ then $\bar{\Gamma} \vdash k : \text{kind}$ and $\bar{\Gamma}, \alpha : k \vdash \tau : \mathbf{T}$.
- If $\Gamma \vdash \sigma_1 \leq \sigma_2 \rightsquigarrow f$ and $\sigma_i \rightsquigarrow [\alpha : k_i, \tau_i]$ then $\bar{\Gamma} \vdash f : \forall \alpha : k_1. \tau_1 \rightarrow \tau_2$.
- If $\Gamma \vdash_P M : \sigma \rightsquigarrow [c, e]$ and $\sigma \rightsquigarrow [\alpha : k, \tau]$ then $\bar{\Gamma} \vdash c : k$ and $\bar{\Gamma} \vdash e : [c/\alpha]\tau$.
- If $\Gamma \vdash_1 M : \sigma \rightsquigarrow e$ and $\sigma \rightsquigarrow [\alpha : k, \tau]$ then $\bar{\Gamma} \vdash e : \exists \alpha : k. \tau$.
- If $\Gamma \vdash e : \tau \rightsquigarrow \bar{e}$ then $\bar{\Gamma} \vdash \bar{e} : \tau$.

4 CONTEXTUAL EQUIVALENCE

We follow Crary [2017] by defining contextual equivalence as the coarsest congruence that is consistent with execution,⁷ and our definitions in this section are adapted from there. However, in contrast to Crary, our indexed relations include a relation on static values. This comes into play in the compatibility rule for polymorphic functions because of the value restriction. Fortunately, this affects the development almost not at all, and ultimately we will prove that the value relation can be ignored (Lemma 4.5).

Definition 4.1. An *indexed dynamic relation* R is a triple of relations: relations (R_t and R_v) on terms and static values indexed by a context and type; and a relation (R_m) on modules indexed by a context, signature, and purity class; such that if $\vdash \Gamma \text{ ok}$ then:

- if $\Gamma \vdash e R_t e' : \tau$ then $\Gamma \vdash e, e' : \tau$, and
if $\Gamma \vdash e R_v e' : \tau$ then $\Gamma \vdash e, e' : \tau$ and e, e' are static values, and
- if $\Gamma \vdash_\kappa M R_m M' : \sigma$ then $\Gamma \vdash_\kappa M, M' : \sigma$, and
- if $\Gamma \vdash_P M R_m M' : \sigma$ then $\Gamma \vdash \text{Fst}(M) \equiv \text{Fst}(M') : \text{Fst}(\sigma)$.

Definition 4.2. Suppose R is an indexed dynamic relation.

- R is *compatible* if it respects the rules in Appendix B. (Informally, R is compatible if R -related expressions can be built from R -related subexpressions.)
- R is *substitutive* if it respects substitution (of values where appropriate) and weakening for constructor, term, and module hypotheses. (For example, if $\Gamma, \alpha/m:\sigma, \Gamma' \vdash e R_t e' : \tau$ and $\Gamma \vdash_P V : \sigma$ then $\Gamma, [c/\alpha]\Gamma' \vdash [\text{Fst}(V), V/\alpha, m]e R_t [\text{Fst}(V), V/\alpha, m]e' : [c/\alpha]\tau$.)
- R is a *congruence* if it is compatible, substitutive, reflexive, transitive, and symmetric.
- R is *consistent* if it preserves module termination. (That is, if $\vdash_I M R_m M' : \sigma$ and M halts then M' halts.)

Although we define consistency in terms of module termination, it is easy to show that a consistent, compatible relation also preserves term termination.

Definition 4.3. *Contextually equivalence* (written $\Gamma \vdash e \approx e' : \tau$ and $\Gamma \vdash e \approx_v e' : \tau$ and $\Gamma \vdash_\kappa M \approx M' : \sigma$) is defined as the union of all consistent congruences.⁸

4.1 Properties of Contextual Equivalence

In proving dynamic correctness of phase separation, we will use several properties of contextual equivalence. One-and-a-half of these can be proven directly (the first and the left-to-right direction of the second). But contextual equivalence, which effectively quantifies over all observing contexts that might use the comparees, is awkward to work with. For the rest of the properties, we require a tool to obtain leverage on contextual equivalence. The tool is logical equivalence, which defines the equivalence of comparees intrinsically, rather than extrinsically through their use in surrounding code.

Crary [2017] defines logical equivalence for the module calculus and shows that it coincides with contextual equivalence. We use that development off-the-shelf, except that we need a notion of logical equivalence for values. We define it as simply the restriction of logical equivalence to static values.

⁷A more common definition would say that two expressions are contextually equivalent if they produce the same behavior when used to fill a hole in a program. Such a definition would be equivalent to ours, but in the setting of the module calculus, it is too unwieldy to use [Crary 2017].

⁸In the Coq formalization, contextual equivalence is given a different definition (the symmetrization of contextual approximation) which is then proved equivalent to this one.

The desired properties are all corollaries of the coincidence of logical and contextual equivalence. Conveniently, we can state them all without reference to logical equivalence, so we do so in the interest of brevity. Proofs are given in the companion Coq formalization.

First, contextual equivalence is adequate for the operational semantics:

LEMMA 4.4 (ADEQUACY). *If $\Gamma \vdash e : \tau$ and $\Gamma \vdash e \mapsto e'$ then $\Gamma \vdash e \approx e' : \tau$, and similarly for modules.*

PROOF SKETCH.

The compatible, reflexive, and transitive closure of evaluation is a consistent congruence, and therefore is contained in contextual equivalence.

Second, contextual equivalence for values is precisely the restriction of ordinary contextual equivalence to static values. Thus, henceforth we can ignore contextual equivalence for values.

LEMMA 4.5 (VALUE EQUIVALENCE). *$\Gamma \vdash sv \approx_v sv' : \tau$ if and only if $\Gamma \vdash sv \approx sv' : \tau$.*

PROOF SKETCH.

Immediate from the definition of logical equivalence and its coincidence with contextual equivalence. Alternatively, for the left-to-right direction, observe that $sv \approx (\Lambda_;1.sv)[\star] \approx (\Lambda_;1.sv')[\star] \approx sv'$.

Third, we can show two expressions are contextually equivalent by considering their closed instances:

LEMMA 4.6 (CLOSED INSTANCES).

- If $\Gamma \vdash e, e' : \tau$ and for any substitution γ such that $\vdash \gamma : \Gamma$ we have $\vdash \gamma(e) \approx \gamma(e') : \gamma(\tau)$, then $\Gamma \vdash e \approx e' : \tau$.
- If $\Gamma \vdash_1 M, M' : \sigma$ and for any substitution γ such that $\vdash \gamma : \Gamma$ we have $\vdash_1 \gamma(M) \approx \gamma(M') : \gamma(\sigma)$, then $\Gamma \vdash_1 M \approx M' : \sigma$.
- If $\Gamma \vdash_P M, M' : \sigma$ and $\Gamma \vdash \text{Fst}(M) \equiv \text{Fst}(M') : \text{Fst}(\sigma)$ and for any substitution γ such that $\vdash \gamma : \Gamma$ we have $\vdash_P \gamma(M) \approx \gamma(M') : \gamma(\sigma)$, then $\Gamma \vdash_P M \approx M' : \sigma$.

Fourth, when two closed expressions have the same termination behavior, we can assume they terminate when showing they are contextually equivalent:

LEMMA 4.7 (CONDITIONAL EQUIVALENCE).

- If $\vdash e, e' : \tau$ and $e \Downarrow \Leftrightarrow e' \Downarrow$, then if $e \Downarrow$ implies $\vdash e \approx e' : \tau$, then $\vdash e \approx e' : \tau$.
- If $\vdash_1 M, M' : \tau$ and $M \Downarrow \Leftrightarrow M' \Downarrow$, then if $M \Downarrow$ implies $\vdash_1 M \approx M' : \sigma$, then $\vdash_1 M \approx M' : \sigma$.
- If $\vdash_P M, M' : \tau$ and $\vdash \text{Fst}(M) \equiv \text{Fst}(M') : \text{Fst}(\sigma)$ and $M \Downarrow \Leftrightarrow M' \Downarrow$, then if $M \Downarrow$ implies $\vdash_P M \approx M' : \sigma$, then $\vdash_P M \approx M' : \sigma$.

Fifth, for closed terms and closed, pure modules, we have a variety of extensionality properties:

LEMMA 4.8 (EXTENSIONALITY). *Suppose $\vdash e, e' : \tau$ and $e \Downarrow \Leftrightarrow e' \Downarrow$, then $\vdash e \approx e' : \tau$ provided:*

- $\tau = \text{unit}$, or
- $\tau = \tau_1 \rightarrow \tau_2$ and for every $\vdash v : \tau_1$ we have $\vdash e v \approx e' v : \tau_2$, or
- $\tau = \forall \alpha:k. \tau'$ and for every $\vdash c : k$ we have $\vdash e[c] \approx e'[c] : [c/\alpha]\tau'$, or
- $\tau = \tau_1 \times \tau_2$ and $\vdash \pi_1 e \approx \pi_1 e' : \tau_1$ and $\vdash \pi_2 e \approx \pi_2 e' : \tau_2$.

Suppose $\vdash_P M, M' : \sigma$ and $M \Downarrow \Leftrightarrow M' \Downarrow$, then $\vdash_P M \approx M' : \sigma$ provided:

- $\sigma = 1$, or
- $\sigma = \langle k \rangle$ and $\vdash \text{Fst}(M) \equiv \text{Fst}(M') : k$, or
- $\sigma = \langle \tau \rangle$ and $\vdash \text{Ext } M \approx \text{Ext } M' : \tau$, or
- $\sigma = \Pi^{\text{gen}} \alpha:\sigma_1. \sigma_2$ and for every $\vdash_P V : \sigma_1$ we have $\vdash_1 MV \approx M'V : [\text{Fst}(V)/\alpha]\sigma_2$, or

$\text{Snd}_1 M$	$\stackrel{\text{def}}{=}$	$\text{let } _ = M \text{ in } \star$
$\text{Snd}_{\langle k \rangle} M$	$\stackrel{\text{def}}{=}$	$\text{let } _ = M \text{ in } \star$
$\text{Snd}_{\langle \tau \rangle} M$	$\stackrel{\text{def}}{=}$	$\text{Ext } M$
$\text{Snd}_{\Pi^{\text{gn}} \alpha : \sigma_1, \sigma_2} M$	$\stackrel{\text{def}}{=}$	$\text{let } m = M$ $\text{in } \Lambda \alpha : k_1. \lambda x : \tau_1. \text{let } \beta / n = m(\text{Join}_{\sigma_1}[\alpha, x])$ $\text{in pack } [\beta, \text{Snd}_{\sigma_2} n] \text{ as } \exists \beta : k_2. \tau_2$ $(\text{where } \sigma_1 \rightsquigarrow [\alpha : k_1. \tau_1] \text{ and } \sigma_2 \rightsquigarrow [\beta : k_2. \tau_2])$
$\text{Snd}_{\Pi^{\text{ap}} \alpha : \sigma_1, \sigma_2} M$	$\stackrel{\text{def}}{=}$	$\text{let } m = M$ $\text{in } \Lambda \alpha : k_1. \lambda x : \tau_1. \text{Snd}_{\sigma_2}(m \cdot (\text{Join}_{\sigma_1}[\alpha, x]))$ $(\text{where } \sigma_1 \rightsquigarrow [\alpha : k_1. \tau_1])$
$\text{Snd}_{\Sigma \alpha : \sigma_1, \sigma_2} M$	$\stackrel{\text{def}}{=}$	$\text{let } \gamma / m = M$ $\text{in } \langle \text{Snd}_{\sigma_1}(\pi_1 m), \text{Snd}_{[\pi_1 \gamma / \alpha] \sigma_2}(\pi_2 m) \rangle$
$\text{Join}_1[c, e]$	$\stackrel{\text{def}}{=}$	$\text{let } _ = e \text{ in } \star$
$\text{Join}_{\langle k \rangle}[c, e]$	$\stackrel{\text{def}}{=}$	$\text{let } _ = e \text{ in } \langle c \rangle$
$\text{Join}_{\langle \tau \rangle}[c, e]$	$\stackrel{\text{def}}{=}$	$\langle e \rangle$
$\text{Join}_{\Pi^{\text{gn}} \alpha : \sigma_1, \sigma_2}[c, e]$	$\stackrel{\text{def}}{=}$	$\text{let } x = e$ $\text{in } \lambda^{\text{gn}} \alpha / m : \sigma_1. \text{unpack } [\beta, y] = x[\alpha](\text{Snd}_{\sigma_1} m)$ $\text{in } (\text{Join}_{\sigma_2}[\beta, y] : \sigma_2)$
$\text{Join}_{\Pi^{\text{ap}} \alpha : \sigma_1, \sigma_2}[c, e]$	$\stackrel{\text{def}}{=}$	$\text{let } x = e$ $\text{in } \lambda^{\text{ap}} \alpha / m : \sigma_1. \text{Join}_{\sigma_2}[c \ \alpha, x[\alpha](\text{Snd}_{\sigma_1} m)]$
$\text{Join}_{\Sigma \alpha : \sigma_1, \sigma_2}[c, e]$	$\stackrel{\text{def}}{=}$	$\text{let } x = e$ $\text{in } \langle \text{Join}_{\sigma_1}[\pi_1 c, \pi_1 x], \text{Join}_{[\pi_1 c / \alpha] \sigma_2}[\pi_2 c, \pi_2 x] \rangle$

Fig. 9. Dynamic Phase Separation

- $\sigma = \Pi^{\text{ap}} \alpha : \sigma_1, \sigma_2$ and $\vdash \text{Fst}(M) \equiv \text{Fst}(M') : \text{Fst}(\Pi^{\text{ap}} \alpha : \sigma_1, \sigma_2)$ and for every $\vdash_P V : \sigma_1$ we have $\vdash_P M \cdot V \approx M' \cdot V : [\text{Fst}(V)/\alpha] \sigma_2$, or
- $\sigma = \Sigma \alpha : \sigma_1, \sigma_2$ and $\vdash_P \pi_1 M \approx \pi_1 M' : \sigma_1$ and $\vdash_P \pi_2 M \approx \pi_2 M' : [\pi_1 \text{Fst}(M)/\alpha] \sigma_2$.

5 CORRECTNESS

Our main aim is to show that every module is contextually equivalent to its translation, in an appropriate sense. But what is the appropriate sense? Since a module and its translation belong to different types, and indeed different syntactic classes, we require some way to relate them.

We do this using Snd_σ and Join_σ , two (families of) functions that convert back-and-forth between modules and terms. Given a module, Snd computes its dynamic component, and given a module's static and dynamic component, Join recovers the original module.

The important fact about Snd_σ and Join_σ is that they are expressed *within the language*. They do not traverse the syntax of their arguments the way the translation does. Rather, they operate through the interface given by their arguments' signature/kind/type. We refer to Snd as *dynamic phase separation* since it does phase separation at run time instead of compile time.

The definitions of Snd and Join are given in Figure 9. The fact that they are definable within the language is interesting in its own right. It shows that modules are a definitional extension of the

core language. In other words, modules confer no fundamental expressive power absent in the core. On the other hand, the fact that the definitions are rather baroque shows that modules are nevertheless a useful feature for a language to support.

LEMMA 5.1. *Suppose $\vdash \Gamma$ ok and $\Gamma \vdash \sigma : \text{sig}$ and $\sigma \rightsquigarrow [\alpha:k. \tau]$. Then:*

- *If $\Gamma \vdash_P M : \sigma$ then $\Gamma \vdash \text{Snd}_\sigma M : [\text{Fst}(M)/\alpha]\tau$.*
- *If $\Gamma \vdash c : k$ and $\Gamma \vdash e : [c/\alpha]\tau$ then $\Gamma \vdash_P \text{Join}_\sigma[c, e] : \sigma$ and $\Gamma \vdash c \equiv \text{Fst}(\text{Join}_\sigma[c, e]) : k$.*

The main lemmas state that Snd and Join are inverses:

LEMMA 5.2 (Join OF Snd). *If $\Gamma \vdash_P M : \sigma$ then $\Gamma \vdash_P M \approx \text{Join}_\sigma[\text{Fst}(M), \text{Snd}_\sigma(M)] : \sigma$.*

PROOF SKETCH.

The proof is by induction on the gross structure of σ (that is, considering all atoms ($\langle k \rangle$ or $\langle \tau \rangle$) to be identical for the purposes of the induction order).

We give one case, by way of example. Suppose $\sigma = \Pi^{\text{ap}} \alpha : \sigma_1. \sigma_2$. By the closed instances lemma, we may assume without loss of generality that Γ is empty. (Note that σ 's instances have the same gross structure as σ .) It is not hard to show that $M \downarrow \Leftrightarrow \text{Join}_\sigma[\text{Fst}(M), \text{Snd}_\sigma(M)] \downarrow$. Therefore, by the conditional equivalence lemma, we may assume without loss of generality that M is some value V . Let $\sigma_1 \rightsquigarrow [\alpha:k_1. \tau_1]$. Then:

$$\begin{aligned}
& \text{Join}_\sigma[\text{Fst}(V), \text{Snd}_\sigma(V)] \\
&= \text{let } f = \text{let } m = V \text{ in } \Lambda \alpha : k_1. \lambda x : \tau_1. \text{Snd}_{\sigma_2}(m \cdot \text{Join}_{\sigma_1}[\alpha, x]) \\
&\quad \text{in } \lambda^{\text{ap}} \alpha / m : \sigma_1. \text{Join}_{\sigma_2}[\text{Fst}(V) \alpha, f[\alpha](\text{Snd}_{\sigma_1} m)] \\
&\approx \text{let } f = \Lambda \alpha : k_1. \lambda x : \tau_1. \text{Snd}_{\sigma_2}(V \cdot \text{Join}_{\sigma_1}[\alpha, x]) \\
&\quad \text{in } \lambda^{\text{ap}} \alpha / m : \sigma_1. \text{Join}_{\sigma_2}[\text{Fst}(V) \alpha, f[\alpha](\text{Snd}_{\sigma_1} m)] \\
&\approx \lambda^{\text{ap}} \alpha / m : \sigma_1. \text{Join}_{\sigma_2}[\text{Fst}(V) \alpha, (\Lambda \alpha : k_1. \lambda x : \tau_1. \text{Snd}_{\sigma_2}(V \cdot \text{Join}_{\sigma_1}[\alpha, x]))[\alpha](\text{Snd}_{\sigma_1} m)] \\
&\approx \lambda^{\text{ap}} \alpha / m : \sigma_1. \text{Join}_{\sigma_2}[\text{Fst}(V) \alpha, \text{Snd}_{\sigma_2}(V \cdot \text{Join}_{\sigma_1}[\alpha, \text{Snd}_{\sigma_1} m])] \\
&= \lambda^{\text{ap}} \alpha / m : \sigma_1. \text{Join}_{\sigma_2}[\text{Fst}(V \cdot m), \text{Snd}_{\sigma_2}(V \cdot \text{Join}_{\sigma_1}[\text{Fst}(m), \text{Snd}_{\sigma_1} m])] \\
&\approx \lambda^{\text{ap}} \alpha / m : \sigma_1. \text{Join}_{\sigma_2}[\text{Fst}(V \cdot m), \text{Snd}_{\sigma_2}(V \cdot m)] \\
&\approx \lambda^{\text{ap}} \alpha / m : \sigma_1. V \cdot m \\
&\approx V
\end{aligned}$$

The last step is by eta equivalence (which follows from extensionality, since V halts), and the previous two steps use the induction hypothesis.

LEMMA 5.3 (Snd OF Join). *If $\Gamma \vdash \sigma : \text{sig}$ and $\sigma \rightsquigarrow [\alpha:k. \tau]$ and $\Gamma \vdash c : k$ and $\Gamma \vdash e : [c/\alpha]\tau$ then $\Gamma \vdash e \approx \text{Snd}_\sigma(\text{Join}_\sigma[c, e]) : [c/\alpha]\tau$.*

PROOF SKETCH.

By induction on the gross structure of σ . (But see Section 5.3.)

Next, we establish the correctness of the subsignature coercion judgement. It is equivalent to the function that converts a term to a module at the subsignature, and back to a term at the supersignature:

LEMMA 5.4. *If $\Gamma \vdash \sigma_1 \leq \sigma_2 \rightsquigarrow f$ and $\sigma_i \rightsquigarrow [\alpha:k_i. \tau_i]$ then $\Gamma \vdash f \approx \Lambda \alpha : k_1. \lambda x : \tau_1. \text{Snd}_{\sigma_2}(\text{Join}_{\sigma_1}[\alpha, x]) : \forall \alpha : k_1. \tau_1 \rightarrow \tau_2$.*

PROOF SKETCH.

By induction on the derivation.

Recall that the variable translation case selfified its signature. To deal with this, we require one more lemma:

LEMMA 5.5. *Suppose $\Gamma \vdash \sigma : \text{sig}$ and $\sigma \rightsquigarrow [\alpha:k. \tau]$ and $\Gamma \vdash c : k$ and $\Gamma \vdash e : [c/\alpha]\tau$. Then $\Gamma \vdash_P \text{Join}_\sigma[c, e] \approx \text{Join}_{S(c:\sigma)}[c, e] : \sigma$.*

PROOF SKETCH.

We can show that if $\Gamma \vdash_P M, M' : \sigma$ and $\Gamma \vdash \text{Fst}(M) \equiv \text{Fst}(M') : \text{Fst}(\sigma)$ and if M and M' differ only in type annotations, then $\Gamma \vdash_P M \approx M' : \sigma$.

Let $S(c : \sigma) \rightsquigarrow [\alpha:k'. \tau']$. Then $k' = S(c : k)$ and we can show that $\Gamma, \alpha:k' \vdash \tau \equiv \tau' : \top$, so $\Gamma \vdash e : [c/\alpha]\tau'$. The first two conditions hold using Lemma 5.1 and $S(c : \sigma) \leq \sigma$. The third is by inspection.

To mediate the difference between the original context Γ and its translation $\bar{\Gamma}$, we define the substitution Snd_Γ to map $\hat{m}$ to $\text{Snd}_\sigma m$ for every $\alpha/m:\sigma \in \Gamma$. Note that Snd_σ does not touch constructor variables, so we need not apply it to any kinds, constructors, or signatures.

LEMMA 5.6. *If $\vdash \Gamma$ ok then $\Gamma \vdash \text{Snd}_\Gamma : \bar{\Gamma}$.*

With this, we can finally establish our main theorem:

THEOREM 5.7 (DYNAMIC CORRECTNESS).

- If $\Gamma \vdash_P M : \sigma \rightsquigarrow [c, e]$ then $\Gamma \vdash M \approx \text{Join}_\sigma[c, \text{Snd}_\Gamma(e)] : \sigma$.
- If $\Gamma \vdash_1 M : \sigma \rightsquigarrow e$ then $\Gamma \vdash_1 M \approx \text{unpack}[\alpha, x] = \text{Snd}_\Gamma(e)$ in $(\text{Join}_\sigma[\alpha, x] : \sigma) : \sigma$.
- If $\Gamma \vdash e : \tau \rightsquigarrow \bar{e}$ then $\Gamma \vdash e \approx \text{Snd}_\Gamma(\bar{e}) : \tau$.

PROOF SKETCH.

The proof is by induction on the translation derivation. We give a few cases by way of example.

Case:

$$\frac{(\alpha/m : \sigma) \in \Gamma}{\Gamma \vdash_P m : S(\alpha : \sigma) \rightsquigarrow [\alpha, \hat{m}]}$$

Then $\text{Join}_{S(\alpha:\sigma)}[\alpha, \text{Snd}_\Gamma(\hat{m})] = \text{Join}_{S(\alpha:\sigma)}[\text{Fst}(m), \text{Snd}_\sigma(m)] \approx \text{Join}_\sigma[\text{Fst}(m), \text{Snd}_\sigma(m)] \approx m$, the second and third steps using Lemmas 5.5 and 5.2.

Case:

$$\frac{\Gamma \vdash \sigma_1 : \text{sig} \quad \Gamma, \alpha/m:\sigma_1 \vdash_P M : \sigma_2 \rightsquigarrow [c, e] \quad \sigma_1 \rightsquigarrow [\alpha:k_1.\tau_1]}{\Gamma \vdash_P \lambda^{\text{ap}}\alpha/m:\sigma_1.M : \Pi^{\text{ap}}\alpha:\sigma_1.\sigma_2 \rightsquigarrow [\lambda\alpha:k_1.c, \Lambda\alpha:k_1.\lambda\hat{m}:\tau_1.e]}$$

Then:

$$\begin{aligned} & \text{Join}_{\Pi^{\text{ap}}\alpha:\sigma_1.\sigma_2}[\lambda\alpha:k_1.c, \text{Snd}_\Gamma(\Lambda\alpha:k_1.\lambda\hat{m}:\tau_1.e)] \\ &= \text{Join}_{\Pi^{\text{ap}}\alpha:\sigma_1.\sigma_2}[\lambda\alpha:k_1.c, \Lambda\alpha:k_1.\lambda\hat{m}:\tau_1.\text{Snd}_\Gamma(e)] \\ &\approx \lambda^{\text{ap}}\alpha/m:\sigma_1.\text{Join}_{\sigma_2}[(\lambda\alpha:k_1.c)\alpha, (\Lambda\alpha:k_1.\lambda\hat{m}:\tau_1.\text{Snd}_\Gamma(e))][\alpha](\text{Snd}_{\sigma_1}m)] \\ &\approx \lambda^{\text{ap}}\alpha/m:\sigma_1.\text{Join}_{\sigma_2}[c, [\text{Snd}_{\sigma_1}m/\hat{m}]\text{Snd}_\Gamma(e)] \\ &= \lambda^{\text{ap}}\alpha/m:\sigma_1.\text{Join}_{\sigma_2}[c, \text{Snd}_{\Gamma, \alpha/m:\sigma_1}(e)] \\ &\approx \lambda^{\text{ap}}\alpha/m:\sigma_1.M \end{aligned}$$

Case:

$$\frac{\Gamma \vdash_P M : \sigma \rightsquigarrow [c, e] \quad \sigma \rightsquigarrow [\alpha:k.\tau]}{\Gamma \vdash_1 M : \sigma \rightsquigarrow \text{pack}[c, e] \text{ as } \exists\alpha:k.\tau}$$

Then $(\text{unpack}[\beta, x] = \text{Snd}_\Gamma(\text{pack}[c, e] \text{ as } \exists\alpha:k.\tau) \text{ in } \text{Join}_\sigma[\beta, x]) = (\text{unpack}[\beta, x] = \text{pack}[c, \text{Snd}_\Gamma(e)] \text{ as } \exists\alpha:k.\tau \text{ in } \text{Join}_\sigma[\beta, x]) \approx \text{Join}_\sigma[c, \text{Snd}_\Gamma(e)] \approx M$.

Case:

$$\frac{\Gamma \vdash_P M : \sigma_1 \rightsquigarrow [c, e] \quad \Gamma \vdash \sigma_1 \leq \sigma_2 \rightsquigarrow f}{\Gamma \vdash_P M : \sigma_2 \rightsquigarrow [c, f[c]e]}$$

Then $\text{Join}_{\sigma_2}[c, \text{Snd}_{\Gamma}(f[c]e)] = \text{Join}_{\sigma_2}[c, f[c](\text{Snd}_{\Gamma}(e))] \approx \text{Join}_{\sigma_2}[c, \text{Snd}_{\sigma_2}(\text{Join}_{\sigma_1}[c, \text{Snd}_{\Gamma}(e)])] \approx \text{Join}_{\sigma_1}[c, \text{Snd}_{\Gamma}(e)] \approx M$. The first step is because f is closed with respect to term variables; the second and third are by Lemmas 5.4 and 5.2.

Observe that for closed terms, the theorem degenerates to if $\vdash e : \tau \rightsquigarrow \bar{e}$ then $\vdash e \approx \bar{e} : \tau$.

5.1 Abstraction Preservation

It remains to collect some corollaries. We can prove a converse of dynamic correctness:

COROLLARY 5.8. *Let Join_{σ} be the substitution that maps m to $\text{Join}_{\sigma}[\alpha, \hat{m}]$ for every $\alpha/m : \sigma \in \Gamma$. (So $\bar{\Gamma} \vdash \text{Join}_{\Gamma} : \Gamma$ whenever $\vdash \Gamma$ ok.) Suppose $\vdash \Gamma$ ok. Then:*

- If $\Gamma \vdash_P M : \sigma \rightsquigarrow [c, e]$ and $\sigma \rightsquigarrow [\alpha:k. \tau]$ then $\bar{\Gamma} \vdash e \approx \text{Snd}_{\sigma}(\text{Join}_{\Gamma}(M)) : [c/\alpha]\tau$.
- If $\Gamma \vdash_1 M : \sigma \rightsquigarrow e$ and $\sigma \rightsquigarrow [\alpha:k. \tau]$ then $\bar{\Gamma} \vdash e \approx \text{let } \alpha/m = \text{Join}_{\Gamma}(M) \text{ in } \text{pack}[\alpha, \text{Snd}_{\sigma}m] \text{ as } \exists \alpha:k. \tau : \exists \alpha:k. \tau$.
- If $\Gamma \vdash e : \tau \rightsquigarrow \bar{e}$ then $\bar{\Gamma} \vdash \bar{e} \approx \text{Join}_{\Gamma}(e) : \tau$.

PROOF SKETCH.

Consider the first clause. Using compatibility, we can show that contextual equivalence respects functionality, so $\text{Join}_{\Gamma}(\text{Snd}_{\Gamma}(e)) \approx e$ by Lemma 5.3. By dynamic correctness, $M \approx \text{Join}_{\sigma}[c, \text{Snd}_{\Gamma}(e)]$. By substitutivity, $\text{Join}_{\Gamma}(M) \approx \text{Join}_{\Gamma}(\text{Join}_{\sigma}[c, \text{Snd}_{\Gamma}(e)]) = \text{Join}_{\sigma}[c, \text{Join}_{\Gamma}(\text{Snd}_{\Gamma}(e))] \approx \text{Join}_{\sigma}[c, e]$. Then, again by Lemma 5.3, $\text{Snd}_{\sigma}(\text{Join}_{\Gamma}(M)) \approx \text{Snd}_{\sigma}(\text{Join}_{\sigma}[c, e]) \approx e$.

The second clause is similar but messier. The third clause is similar but simpler.

Abstraction preservation follows directly from this. If one neglects (for the moment) the difference between the source and target language, this is the hard direction of full abstraction:

COROLLARY 5.9 (ABSTRACTION PRESERVATION). *Suppose $\vdash \Gamma$ ok. Then:*

- If $\Gamma \vdash_P M_1 \approx M_2 : \sigma$ and $\sigma \rightsquigarrow [\alpha:k. \tau]$ and $\Gamma \vdash_P M_i : \sigma \rightsquigarrow [c_i, e_i]$ then $\bar{\Gamma} \vdash e_1 \approx e_2 : [c_1/\alpha]\tau$.
- If $\Gamma \vdash_1 M_1 \approx M_2 : \sigma$ and $\sigma \rightsquigarrow [\alpha:k. \tau]$ and $\Gamma \vdash_1 M_i : \sigma \rightsquigarrow e_i$ then $\bar{\Gamma} \vdash e_1 \approx e_2 : \exists \alpha:k. \tau$.
- If $\Gamma \vdash e_1 \approx e_2 : \tau$ and $\Gamma \vdash e_i : \tau \rightsquigarrow \bar{e}_i$ then $\bar{\Gamma} \vdash \bar{e}_1 \approx \bar{e}_2 : \tau$.

PROOF SKETCH.

We give the first clause; the others are similar. Suppose $\Gamma \vdash_P M_1 \approx M_2 : \sigma$. By substitutivity of contextual equivalence, $\bar{\Gamma} \vdash \text{Join}_{\Gamma}(M_1) \approx \text{Join}_{\Gamma}(M_2) : \sigma$. Therefore $e_1 \approx \text{Snd}_{\sigma}(\text{Join}_{\Gamma}(M_1)) \approx \text{Snd}_{\sigma}(\text{Join}_{\Gamma}(M_2)) \approx e_2$ by Corollary 5.8 and compatibility.

The coherence of the translation is an immediate consequence of this, using reflexivity for the antecedent:

COROLLARY 5.10 (COHERENCE). *Suppose $\vdash \Gamma$ ok. Then:*

- If $\Gamma \vdash_P M : \sigma$ and $\sigma \rightsquigarrow [\alpha:k. \tau]$ and $\Gamma \vdash_P M : \sigma \rightsquigarrow [c_i, e_i]$ (for $i = 1, 2$) then $\bar{\Gamma} \vdash e_1 \approx e_2 : [c_1/\alpha]\tau$.
- If $\Gamma \vdash_1 M : \sigma$ and $\sigma \rightsquigarrow [\alpha:k. \tau]$ and $\Gamma \vdash_1 M : \sigma \rightsquigarrow e_i$ (for $i = 1, 2$) then $\bar{\Gamma} \vdash e_1 \approx e_2 : \exists \alpha:k. \tau$.
- If $\Gamma \vdash e : \tau$ and $\Gamma \vdash e : \tau \rightsquigarrow \bar{e}_i$ (for $i = 1, 2$) then $\bar{\Gamma} \vdash \bar{e}_1 \approx \bar{e}_2 : \tau$.

5.2 Full Abstraction

Full abstraction states that expressions are source-equivalent (*i.e.*, equivalent in the source language) exactly when their translations are target-equivalent. Abstraction preservation gives us most of this: It says that translations of source-equivalent expressions are source-equivalent. Since the target language is a subset of the source, every observing context in the target is also one in the source, and consequently source-equivalence implies target-equivalence. Thus, abstraction preservation gives us half of full abstraction.

It remains to show that expressions are source-equivalent when their translations are target-equivalent. Dynamic correctness says that source expressions are source-equivalent to their translations, so the result follows by transitivity, provided target-equivalent expressions are source-equivalent:

LEMMA 5.11. *Let $\Gamma \vdash_{\text{tgt}} e \approx e' : \tau$ be contextual equivalence for the target, defined analogously to Section 4, but using only the features of the target language. Then $\Gamma \vdash_{\text{tgt}} e \approx e' : \tau$ implies $\Gamma \vdash e \approx e' : \tau$.*

PROOF SKETCH.

As developed in Crary [2017], contextual equivalence coincides with logical equivalence, which is biorthogonally closed [Pitts 2005]. Dynamic correctness tells us that every closed source term has an equivalent target term. Thus, in biorthogonal closure we may assume without loss of generality that the continuations are target terms. It follows that biorthogonally closed sets are closed under target-equivalence.

COROLLARY 5.12 (FULL ABSTRACTION). *Suppose $\vdash \Gamma \text{ ok}$. Then:*

- *If $\sigma \rightsquigarrow [\alpha:k. \tau]$ and $\Gamma \vdash_P M_i : \sigma \rightsquigarrow [c_i, e_i]$ then $\Gamma \vdash_P M_1 \approx M_2 : \sigma$ exactly when both $\bar{\Gamma} \vdash c_1 \equiv c_2 : k$ and $\bar{\Gamma} \vdash_{\text{tgt}} e_1 \approx e_2 : [c_1/\alpha]\tau$.*
- *If $\sigma \rightsquigarrow [\alpha:k. \tau]$ and $\Gamma \vdash_I M_i : \sigma \rightsquigarrow e_i$ then $\Gamma \vdash_I M_1 \approx M_2 : \sigma$ exactly when $\bar{\Gamma} \vdash_{\text{tgt}} e_1 \approx e_2 : \exists \alpha:k. \tau$.*
- *If $\Gamma \vdash e_i : \tau \rightsquigarrow e'_i$ then $\Gamma \vdash e_1 \approx e_2 : \tau$ exactly when $\bar{\Gamma} \vdash_{\text{tgt}} e'_1 \approx e'_2 : \tau$.*

5.3 The Role of the Value Restriction

Observe that $\text{Join} \circ \text{Snd}$ and $\text{Snd} \circ \text{Join}$ have the effect of eta-expanding their arguments (after evaluating them to values). For example, suppose F is an applicative functor value. As we saw in the proof sketch of Lemma 5.2, $\text{Join}[\text{Fst}(F), \text{Snd}(F)]$ is equivalent, after some simplification, to $\lambda m. F \cdot m$. The latter is equivalent to F because F is a value.

On the other hand, suppose f is a value, and suppose c and f have an appropriate kind/type to be the static and dynamic components of an applicative functor. Note that f 's type must have the form $\forall \alpha:k. \tau \rightarrow \tau'$. Then $\text{Snd}(\text{Join}[c, f])$ is equivalent, after some simplification, to $\Lambda \alpha. \lambda x. f[\alpha]x$. This is equivalent to f , but only because of the value restriction. Since f is a value, it is certainly equivalent to $\Lambda \alpha. f[\alpha]$, but $f[\alpha]$ is *not* a value, so without a value restriction, it would not in general be equivalent to $\lambda x. f[\alpha]x$. The value restriction ensures that $f[\alpha]$ will halt, which is good enough to conclude $f[\alpha] \approx \lambda x. f[\alpha]x$. Thus Lemma 5.3 relies crucially on the value restriction.

We can work backward from this observation to obtain a counterexample of full abstraction if the value restriction were omitted. Suppose m is a variable with an applicative-functor signature. Since m is a value, $m \approx \lambda^{\text{ap}} \alpha/n. m \cdot n$. But m translates to dynamic component $\hat{m}$, whereas $\lambda^{\text{ap}} \alpha/n. m \cdot n$ translates to dynamic component $\Lambda \alpha. \lambda \hat{n}. \hat{m}[\alpha]\hat{n}$. These translations can be distinguished by the observing context $\text{let } \hat{m} = \Lambda \alpha. \perp \text{ in let } _ = [\] c \text{ in } \star$ (where c is any constructor of appropriate kind).

This issue with full abstraction does not rely on applicative functors. A similar example can be constructed using a generative functor.

(Both Shan [2006] and Rossberg, *et al.* [2014] produce a generally similar translation to ours, but neither of their target languages has a value restriction on polymorphic functions.⁹ Consequently neither of them preserves abstraction, so long as their target language is capable of expressing the eager let used in the observing context above.)

Note that this issue does not rule out having a second form of polymorphic function without a value restriction, were one desired. The important thing is that the target language have a form of function able to take in both a constructor and a value at once, without any intervening effects.

One could even have a value restriction in the target and not the source, although having different source and target languages would complicate the development. We also conjecture that the translation would be fully abstract even without a value restriction if it were to translate variables to their full eta-long form. But such a translation is undesirable for efficiency reasons, so we have not explored it carefully.

6 DEFINITIONAL EXTENSIONS

The alignment of abstraction with effects in our module calculus suggests a natural treatment of first-class modules [Dreyer 2005; Russo 2000] as a definitional extension. Suppose we have a new term form $\text{md}_\sigma(M)$, which stands for an encapsulated module and has type $\text{md}(\sigma)$. And suppose such an encapsulated module can be made back into a module using $\text{tm}_\sigma(e)$.

Although this formulation offers less expressive power than systems that allow the propagation of type information through first-class code [Harper and Lillibridge 1994], it supports most important applications of first-class modules such as run-time selection of implementations of abstract data types.

The critical point in the static semantics of first-class modules is that the tm form must be impure. Since the module depends on arbitrary code, it is impossible to determine its type components statically:

$$\frac{\Gamma \vdash_1 M : \sigma}{\Gamma \vdash \text{md}_\sigma(M) : \text{md}(\sigma)} \quad \frac{\Gamma \vdash e : \text{md}(\sigma)}{\Gamma \vdash_1 \text{tm}_\sigma(e) : \sigma}$$

Two questions present themselves regarding these first-class modules: Does the language still respect the phase distinction, and does the extension leave intact the language's existing abstraction properties? At first glance, neither seems certain. The phase distinction depends on the fact that types cannot depend on terms, but first-class modules appear, at least superficially, to provide a way for that to happen. And, first-class modules provide a wealth of new observing contexts that might be used to distinguish between previously indistinguishable modules.

But the machinery developed here allows us easily to answer both questions in the affirmative. We show that we can faithfully define first-class modules as a definitional extension in the original language. This shows that the extension respects the phase distinction, since the original did. It also shows that the extension preserves abstraction, because any observing context in the extended language can be turned into an equivalent observing context in the original language. Consequently, any modules that are indistinguishable in the original language remain so with the extension.

Suppose $\sigma \rightsquigarrow [\alpha:k. \tau]$. We define:

$$\begin{aligned} \text{md}(\sigma) &\stackrel{\text{def}}{=} \exists \alpha:k. \tau \\ \text{md}_\sigma(M) &\stackrel{\text{def}}{=} \text{let } \alpha/m = M \text{ in pack } [\alpha, \text{Snd}_\sigma m] \text{ as } \exists \alpha:k. \tau \\ \text{tm}_\sigma(e) &\stackrel{\text{def}}{=} \text{unpack } [\alpha, x] = e \text{ in } (\text{Join}_\sigma [\alpha, x] : \sigma) \end{aligned}$$

⁹Note that it is the *target* language at issue here, not any value restriction that might exist in the source language. Both Shan and Rossberg *et al.* target ordinary F_ω (extended with extra types), which has no value restriction.

We can easily show that $\text{tm}_\sigma(\text{md}_\sigma(N)) \approx N$, thereby simulating the extension's operational semantics.

Another useful extension is an operation $\text{purify}_{S(c:\sigma)}(M)$ that takes an impure but transparent module and makes it pure [Dreyer 2005; Shao 1999]. This expresses the fact that in a module whose exported type components are fully specified, any typing effects are benign. This is useful for using generative functors within the implementation of an applicative functor.

$$\frac{\Gamma \vdash c : \text{Fst}(\sigma) \quad \Gamma \vdash_! M : S(c : \sigma)}{\Gamma \vdash_P \text{purify}_{S(c:\sigma)}(M) : S(c : \sigma)}$$

We show that purify is a definitional extension by defining $\text{purify}_{S(c:\sigma)}(M)$ as:

$$\text{Join}_{S(c:\sigma)}[c, \text{let } m = M \text{ in } \text{Snd}_{S(c:\sigma)}m]$$

This trick has two parts: First, by converting M to a term, we disappear any typing effects it might have. Then, since M is transparent, its static component (c) is known without referring to M , so we are able to convert back to a pure module using Join .

When M is a value (and hence pure), we can easily show that $\text{purify}_{S(c:\sigma)}(M) \approx M$, thereby simulating the extension's operational semantics.

Although these definitions are satisfactory from a theoretical standpoint, in practice it may be preferable to translate the extensions directly:

$$\frac{\Gamma \vdash_! M : \sigma \rightsquigarrow e}{\Gamma \vdash \text{md}_\sigma(M) : \text{md}(\sigma) \rightsquigarrow e} \quad \frac{\Gamma \vdash e : \text{md}(\sigma) \rightsquigarrow \bar{e}}{\Gamma \vdash_! \text{tm}_\sigma(e) : \sigma \rightsquigarrow \bar{e}}$$

Observe that first-class modules are actually trivial when viewed through the lens of phase separation. On the other hand, the type translation is no longer the identity ($\text{md}(\sigma)$ becomes $\exists \alpha : k.\tau$), which adds a bit of overhead to the compiler.

$$\frac{\Gamma \vdash c : \text{Fst}(\sigma) \quad \Gamma \vdash_! M : S(c : \sigma) \rightsquigarrow e}{\Gamma \vdash_P \text{purify}_{S(c:\sigma)}(M) : S(c : \sigma) \rightsquigarrow [c, \text{unpack}[\alpha, x] = e \text{ in } x]}$$

This translation works because in the unpack , α has the kind $S(c : \text{Fst}(\sigma))$, so $\alpha \equiv c : \text{Fst}(\sigma)$, so x 's type can be expressed without reference to α .

7 FORMALIZATION

All the results in this paper are formalized in Coq (version 8.4). We implemented binding using deBruijn indices and explicit substitutions. To make reasoning about relations cleaner, we used the axioms of functional and propositional extensionality.

The full development is 89k lines. (These counts include comments and whitespace.) Of those, 70k are preliminaries or the development of logical equivalence, which are borrowed from Crary [2017], with some modest editing, mostly to deal with the value restriction. The all-new material—the translation and its correctness proof—account for 19k lines.

We remarked above that the addition of value restriction had almost no effect on the development of logical equivalence. As a coarse measure of that, updating the logical-equivalence Coq proofs to account for the value restriction took less than one day.

The numerous typing premises involved in showing contextual equivalences rapidly become unmanageable. To manage this, we defined *reductive contextual equivalence*. We say that $\Gamma \vdash e \Rightarrow e'$ if for any τ , $\Gamma \vdash e : \tau$ implies $\Gamma \vdash e \approx e' : \tau$ (and similarly for modules). One example of how this simplifies proofs is we can say that $\Gamma \vdash e \mapsto e'$ implies $\Gamma \vdash e \Rightarrow e'$, eliminating the typing premise from Lemma 4.4.

8 ADDITIONAL DISCUSSION OF RELATED WORK

As discussed above, the closest work to ours is Shan [2006] and Rossberg, *et al.* [2014]. The most significant difference from ours is they do not establish dynamic correctness, but there is a stylistic difference as well. Although they are not fundamentally different from the pattern of the Harper, *et al.* [1990], their translations do not explicitly separate modules into type and term components. Instead, they realize the phase distinction by separating all the opaque type components into a prefix on which the module depends. (Shao [1999] calls this prefix the module's *flexroot*.) In ordinary modules, the prefix is existentially quantified, and in functors it is lambda abstracted. In essence, the module is automatically rewritten to replace sharing-by-fibration with sharing-by-parameterization [Harper and Pierce 2005]. An advantage of their presentation is it allows one to dispense with any machinery to control the propagation of type information (such as the singleton kinds we use here).

These two differences may be connected. Our proof technique relies crucially on separating modules into type constructors and terms. It is not clear how to define something like `Snd` and `Join` without doing so.

Shan also addresses weak sealing [Dreyer *et al.* 2003], which we do not. He does so by generating a two-stage program. The first stage resolves static effects (in the sense of Dreyer, *et al.* [2003]) that arise from weak sealing, resulting in a program without weak sealing. The second stage then executes that program, resolving the remaining (dynamic) effects as it goes. Rossberg, *et al.* do something similar. This approach is very plausible, but we do not follow it here, because the equational theory of weak sealing is still not well understood. Specifically, we do not yet know how to reconcile logical equivalence with weak sealing, so we cannot even begin to prove full abstraction.

Elsman, *et al.* [1999; 2018] suggest a radically different approach called static interpretation. It resolves all module-level computation (*e.g.*, functor application) at compile time. This sacrifices separate compilation (functors cannot be compiled separately from their arguments), but it makes it easy to optimize across module boundaries. In essence, static interpretation weakens the phase distinction by assigning module-level computation to the compile-time phase rather than the run-time phase. Their languages do not include sealing, but do include generative datatypes, which offer a means of abstraction. As with other prior work, they do not show dynamic correctness.

9 CONCLUSION

Our correctness and full abstraction results show that the phase-separation algorithm is canonical, from a theoretical standpoint anyway. Any other algorithm that is dynamically correct (in our sense) is necessarily equivalent to ours.

Nevertheless, there is room for improvement in the language itself. Our module calculus supports only a fragment of the features needed for Standard ML [Lee *et al.* 2007]. Many of the missing features are easily supported, but some—particularly those involving references, exceptions, or other forms of effects—are more difficult. Another important feature for languages that emphasize applicative functors (such as OCaml [Leroy *et al.* 2016]) is weak sealing [Dreyer *et al.* 2003]. Still another direction is more powerful module features such as recursive modules [Crary *et al.* 1999; Dreyer 2005; Russo 2001].

In each of these cases, we anticipate that the general proof strategy employed here should work. However, a precondition for progress in any of these domains is a working account of logical equivalence. These are important avenues for future work.

A THE TERM AND SUBSIGNATURE TRANSLATIONS

$$\boxed{\Gamma \vdash e : \tau \rightsquigarrow \bar{e}}$$

$$\frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau \rightsquigarrow x} \quad \frac{}{\Gamma \vdash \star : \text{unit} \rightsquigarrow \star}$$

$$\frac{\Gamma \vdash \tau_1 : T \quad \Gamma, x:\tau_1 \vdash e : \tau_2 \rightsquigarrow \bar{e}}{\Gamma \vdash \lambda x:\tau_1. e : \tau_1 \rightarrow \tau_2 \rightsquigarrow \lambda x:\tau_1. \bar{e}} \quad \frac{\Gamma \vdash e_1 : \tau \rightarrow \tau' \rightsquigarrow \bar{e}_1 \quad \Gamma \vdash e_2 : \tau \rightsquigarrow \bar{e}_2}{\Gamma \vdash e_1 e_2 : \tau' \rightsquigarrow \bar{e}_1 \bar{e}_2}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \rightsquigarrow \bar{e}_1 \quad \Gamma \vdash e_2 : \tau_2 \rightsquigarrow \bar{e}_2}{\Gamma \vdash \langle e_1, e_2 \rangle : \tau_1 \times \tau_2 \rightsquigarrow \langle \bar{e}_1, \bar{e}_2 \rangle} \quad \frac{\Gamma \vdash e : \tau_1 \times \tau_2 \rightsquigarrow \bar{e}}{\Gamma \vdash \pi_1 e : \tau_1 \rightsquigarrow \pi_1 \bar{e}} \quad \frac{\Gamma \vdash e : \tau_1 \times \tau_2 \rightsquigarrow \bar{e}}{\Gamma \vdash \pi_2 e : \tau_2 \rightsquigarrow \pi_2 \bar{e}}$$

$$\frac{\Gamma \vdash k : \text{kind} \quad \Gamma, \alpha:k \vdash sv : \tau \rightsquigarrow \bar{e}}{\Gamma \vdash \Lambda \alpha:k. sv : \forall \alpha:k. \tau \rightsquigarrow \Lambda \alpha:k. \bar{e}} \quad \frac{\Gamma \vdash e : \forall \alpha:k. \tau \rightsquigarrow \bar{e} \quad \Gamma \vdash c : k}{\Gamma \vdash e[c] : [c/\alpha] \tau \rightsquigarrow \bar{e}[c]}$$

$$\frac{\Gamma \vdash c : k \quad \Gamma \vdash e : [c/\alpha] \tau \rightsquigarrow \bar{e} \quad \Gamma, \alpha:k \vdash \tau : T}{\Gamma \vdash \text{pack}[c, e] \text{ as } \exists \alpha:k. \tau : \exists \alpha:k. \tau \rightsquigarrow \text{pack}[c, \bar{e}] \text{ as } \exists \alpha:k. \tau}$$

$$\frac{\Gamma \vdash e_1 : \exists \alpha:k. \tau \rightsquigarrow \bar{e}_1 \quad \Gamma, \alpha:k, x:\tau \vdash e_2 : \tau' \rightsquigarrow \bar{e}_2 \quad \Gamma \vdash \tau' : T}{\Gamma \vdash \text{unpack}[\alpha, x] = e_1 \text{ in } e_2 : \tau' \rightsquigarrow \text{unpack}[\alpha, x] = \bar{e}_1 \text{ in } \bar{e}_2}$$

$$\frac{\Gamma \vdash e : (\text{unit} \rightarrow \tau) \rightarrow \tau \rightsquigarrow \bar{e}}{\Gamma \vdash \text{fix}_\tau e : \tau \rightsquigarrow \text{fix}_\tau \bar{e}} \quad \frac{\Gamma \vdash e_1 : \tau_1 \rightsquigarrow \bar{e}_1 \quad \Gamma, x:\tau_1 \vdash e_2 : \tau_2 \rightsquigarrow \bar{e}_2}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau_2 \rightsquigarrow \text{let } x = \bar{e}_1 \text{ in } \bar{e}_2}$$

$$\frac{\Gamma \vdash_1 M : \sigma \rightsquigarrow e_1 \quad \Gamma, \alpha/m:\sigma \vdash e : \tau \rightsquigarrow e_2 \quad \Gamma \vdash \tau : T}{\Gamma \vdash \text{let } \alpha/m = M \text{ in } e : \tau \rightsquigarrow \text{unpack}[\alpha, \hat{m}] = e_1 \text{ in } e_2}$$

$$\frac{\Gamma \vdash_1 M : \langle \tau \rangle \rightsquigarrow e}{\Gamma \vdash \text{Ext } M : \tau \rightsquigarrow \text{unpack}[_, x] = e \text{ in } x} \quad \frac{\Gamma \vdash e : \tau \rightsquigarrow \bar{e} \quad \Gamma \vdash \tau \equiv \tau' : T}{\Gamma \vdash e : \tau' \rightsquigarrow \bar{e}}$$

$$\boxed{\Gamma \vdash \sigma \leq \sigma' \rightsquigarrow f}$$

$$\frac{\Gamma \vdash \sigma \equiv \sigma' : \text{sig} \quad \sigma \rightsquigarrow [\alpha:k. \tau]}{\Gamma \vdash \sigma \leq \sigma' \rightsquigarrow \Lambda \alpha:k. \lambda x:\tau. x} \quad \frac{\Gamma \vdash \sigma \leq \sigma' \rightsquigarrow f \quad \Gamma \vdash \sigma' \leq \sigma'' \rightsquigarrow f' \quad \sigma \rightsquigarrow [\alpha:k. \tau]}{\Gamma \vdash \sigma \leq \sigma'' \rightsquigarrow \Lambda \alpha:k. \lambda x:\tau. f'[\alpha](f[\alpha]x)}$$

$$\frac{\Gamma \vdash k \leq k'}{\Gamma \vdash \langle k \rangle \leq \langle k' \rangle \rightsquigarrow \Lambda \alpha:k. \lambda x:\text{unit}. x}$$

$$\frac{\Gamma \vdash \sigma'_1 \leq \sigma_1 \rightsquigarrow f_1 \quad \Gamma, \alpha:\text{Fst}(\sigma'_1) \vdash \sigma_2 \leq \sigma'_2 \rightsquigarrow f_2 \quad \Gamma, \alpha:\text{Fst}(\sigma_1) \vdash \sigma_2 : \text{sig} \quad \sigma_1 \rightsquigarrow [\alpha:k_1. \tau_1] \quad \sigma'_1 \rightsquigarrow [\alpha:k'_1. \tau'_1] \quad \sigma_2 \rightsquigarrow [\beta:k_2. \tau_2] \quad \sigma'_2 \rightsquigarrow [\beta:k'_2. \tau'_2]}{\Gamma \vdash \Pi^{\text{gn}} \alpha:\sigma_1. \sigma_2 \leq \Pi^{\text{gn}} \alpha:\sigma'_1. \sigma'_2 \rightsquigarrow \Lambda _ : 1. \lambda f. (\forall \alpha:k_1. \tau_1 \rightarrow \exists \beta:k_2. \tau_2). \Lambda \alpha:k'_1. \lambda x:\tau'_1. \text{unpack}[\beta, y] = f[\alpha](f_1[\alpha]x) \text{ in pack}[\beta, f_2[\beta]y] \text{ as } \exists \beta:k'_2. \tau'_2}$$

$$\begin{array}{c}
\frac{\Gamma \vdash \sigma'_1 \leq \sigma_1 \rightsquigarrow f_1 \quad \Gamma, \alpha: \text{Fst}(\sigma'_1) \vdash \sigma_2 \leq \sigma'_2 \rightsquigarrow f_2}{\Gamma, \alpha: \text{Fst}(\sigma_1) \vdash \sigma_2 : \text{sig} \quad \sigma_1 \rightsquigarrow [\alpha:k_1.\tau_1] \quad \sigma'_1 \rightsquigarrow [\alpha:k'_1.\tau'_1] \quad \sigma_2 \rightsquigarrow [\beta:k_2.\tau_2]} \\
\frac{\Gamma \vdash \Pi^{\text{ap}} \alpha: \sigma_1.\sigma_2 \leq \Pi^{\text{ap}} \alpha: \sigma'_1.\sigma'_2}{\sim \Lambda \gamma: (\Pi \alpha: k_1.k_2). \lambda f: (\forall \alpha: k_1.\tau_1 \rightarrow [\gamma \alpha / \beta] \tau_2).} \\
\Lambda \alpha: k'_1. \lambda x: \tau'_1. f_2 [\gamma \alpha] (f [\alpha] (f_1 [\alpha] x)) \\
\Gamma \vdash \sigma_1 \leq \sigma'_1 \rightsquigarrow f_1 \quad \Gamma, \alpha: \text{Fst}(\sigma_1) \vdash \sigma_2 \leq \sigma'_2 \rightsquigarrow f_2 \quad \Gamma, \alpha: \text{Fst}(\sigma'_1) \vdash \sigma'_2 : \text{sig} \\
\sigma_1 \rightsquigarrow [\alpha:k_1.\tau_1] \quad \sigma_2 \rightsquigarrow [\beta:k_2.\tau_2]} \\
\frac{}{\Gamma \vdash \Sigma \alpha: \sigma_1.\sigma_2 \leq \Sigma \alpha: \sigma'_1.\sigma'_2} \\
\sim \Lambda \gamma: (\Sigma \alpha: k_1.k_2). \lambda x: ([\pi_1 \gamma / \alpha] \tau_1 \times [\pi_1 \gamma, \pi_2 \gamma / \alpha, \beta] \tau_2). \\
\langle f_1 [\pi_1 \gamma] (\pi_1 x), ([\pi_1 \gamma / \alpha] f_2) [\pi_2 \gamma] (\pi_2 x) \rangle
\end{array}$$

B COMPATIBILITY

$$\begin{array}{c}
\frac{\Gamma \vdash \tau_1 : \mathbb{T} \quad \Gamma, x: \tau_1 \vdash e R_t e' : \tau_2}{\Gamma \vdash \lambda x: \tau_1. e R_t \lambda x: \tau_1. e' : \tau_1 \rightarrow \tau_2} \quad \frac{\Gamma \vdash e_1 R_t e'_1 : \tau \rightarrow \tau' \quad \Gamma \vdash e_2 R_t e'_2 : \tau}{\Gamma \vdash e_1 e_2 R_t e'_1 e'_2 : \tau'} \\
\frac{\Gamma \vdash e_1 R_t e'_1 : \tau_1 \quad \Gamma \vdash e_2 R_t e'_2 : \tau_2}{\Gamma \vdash \langle e_1, e_2 \rangle R_t \langle e'_1, e'_2 \rangle : \tau_1 \times \tau_2} \quad \frac{\Gamma \vdash e R_t e' : \tau_1 \times \tau_2}{\Gamma \vdash \pi_1 e R_t \pi_1 e' : \tau_1} \quad \frac{\Gamma \vdash e R_t e' : \tau_1 \times \tau_2}{\Gamma \vdash \pi_2 e R_t \pi_2 e' : \tau_2} \\
\frac{\Gamma \vdash k : \text{kind} \quad \Gamma, \alpha: k; \Gamma \vdash e R_v e' : \tau}{\Gamma \vdash \Lambda \alpha: k. e R_t \Lambda \alpha: k. e' : \forall \alpha: k. \tau} \quad \frac{\Gamma \vdash e R_t e' : \forall \alpha: k. \tau \quad \Gamma \vdash c : k}{\Gamma \vdash e[c] R_t e'[c] : [c/\alpha] \tau} \\
\frac{\Gamma \vdash c : k \quad \Gamma \vdash e R_t e' : [c/\alpha] \tau \quad \Gamma, \alpha: k \vdash \tau : \mathbb{T}}{\Gamma \vdash \text{pack}[c, e] \text{ as } \exists \alpha: k. \tau R_t \text{pack}[c, e'] \text{ as } \exists \alpha: k. \tau : \exists \alpha: k. \tau} \\
\frac{\Gamma \vdash e_1 R_t e'_1 : \exists \alpha: k. \tau \quad \Gamma, \alpha: k, x: \tau \vdash e_2 R_t e'_2 : \tau' \quad \Gamma \vdash \tau' : \mathbb{T}}{\Gamma \vdash \text{unpack}[\alpha, x] = e_1 \text{ in } e_2 R_t \text{unpack}[\alpha, x] = e'_1 \text{ in } e'_2 : \tau'} \\
\frac{\Gamma \vdash \tau : \mathbb{T} \quad \Gamma \vdash e R_t e' : (\text{unit} \rightarrow \tau) \rightarrow \tau}{\Gamma \vdash \text{fix}_\tau e R_t \text{fix}_\tau e' : \tau} \quad \frac{\Gamma \vdash e_1 R_t e'_1 : \tau_1 \quad \Gamma, x: \tau_1 \vdash e_2 R_t e'_2 : \tau_2}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 R_t \text{let } x = e'_1 \text{ in } e'_2 : \tau_2} \\
\frac{\Gamma \vdash M R_m M' : \sigma \quad \Gamma, \alpha/m: \sigma \vdash e R_t e' : \tau \quad \Gamma \vdash \tau : \mathbb{T}}{\Gamma \vdash \text{let } \alpha/m = M \text{ in } e R_t \text{let } \alpha/m = M' \text{ in } e' : \tau} \\
\frac{\Gamma \vdash M R_m M' : \langle \tau \rangle}{\Gamma \vdash \text{Ext } M R_t \text{Ext } M' : \tau} \quad \frac{\Gamma \vdash e R_t e' : \tau \quad \Gamma \vdash \tau \equiv \tau' : \mathbb{T}}{\Gamma \vdash e R_t e' : \tau'} \\
\frac{\Gamma \vdash \tau_1 : \mathbb{T} \quad \Gamma, x: \tau_1 \vdash e R_t e' : \tau_2}{\Gamma \vdash \lambda x: \tau_1. e R_v \lambda x: \tau_1. e' : \tau_1 \rightarrow \tau_2} \quad \frac{\Gamma \vdash e_1 R_v e'_1 : \tau_1 \quad \Gamma \vdash e_2 R_v e'_2 : \tau_2}{\Gamma \vdash \langle e_1, e_2 \rangle R_v \langle e'_1, e'_2 \rangle : \tau_1 \times \tau_2} \\
\frac{\Gamma \vdash k : \text{kind} \quad \Gamma, \alpha: k; \Gamma \vdash e R_v e' : \tau}{\Gamma \vdash \Lambda \alpha: k. e R_v \Lambda \alpha: k. e' : \forall \alpha: k. \tau} \\
\frac{\Gamma \vdash c : k \quad \Gamma \vdash e R_v e' : [c/\alpha] \tau \quad \Gamma, \alpha: k \vdash \tau : \mathbb{T}}{\Gamma \vdash \text{pack}[c, e] \text{ as } \exists \alpha: k. \tau R_v \text{pack}[c, e'] \text{ as } \exists \alpha: k. \tau : \exists \alpha: k. \tau} \quad \frac{\Gamma \vdash e R_v e' : \tau \quad \Gamma \vdash \tau \equiv \tau' : \mathbb{T}}{\Gamma \vdash e R_v e' : \tau'} \\
\frac{\Gamma \vdash c : k}{\Gamma_P \vdash \langle c \rangle R_m \langle c \rangle : \langle k \rangle} \quad \frac{\Gamma \vdash e R_t e' : \tau}{\Gamma_P \vdash \langle e \rangle R_m \langle e' \rangle : \langle \tau \rangle} \\
\frac{\Gamma \vdash \sigma_1 : \text{sig} \quad \Gamma, \alpha/m: \sigma_1 \vdash_1 M R_m M' : \sigma_2}{\Gamma_P \vdash \lambda^{\text{gn}} \alpha/m: \sigma_1. M R_m \lambda^{\text{gn}} \alpha/m: \sigma_1. M' : \Pi^{\text{gn}} \alpha: \sigma_1. \sigma_2} \\
\frac{\Gamma \vdash_1 M_1 R_m M'_1 : \Pi^{\text{gn}} \alpha: \sigma. \sigma' \quad \Gamma_P \vdash M_2 R_m M'_2 : \sigma \quad \Gamma \vdash \text{Fst}(M_2) \gg c_2}{\Gamma \vdash_1 M_1 M_2 R_m M'_1 M'_2 : [c_2/\alpha] \sigma'} \\
\frac{\Gamma \vdash \sigma_1 : \text{sig} \quad \Gamma, \alpha/m: \sigma_1 \vdash_P M R_m M' : \sigma_2}{\Gamma_P \vdash \lambda^{\text{ap}} \alpha/m: \sigma_1. M R_m \lambda^{\text{ap}} \alpha/m: \sigma_1. M' : \Pi^{\text{ap}} \alpha: \sigma_1. \sigma_2}
\end{array}$$

$$\begin{array}{c}
\frac{\Gamma \vdash_{\kappa} M_1 R_m M'_1 : \Pi^{\text{ap}} \alpha : \sigma . \sigma' \quad \Gamma \vdash_{\text{P}} M_2 R_m M'_2 : \sigma \quad \Gamma \vdash \text{Fst}(M_2) \gg c_2}{\Gamma \vdash_{\kappa} M_1 \cdot M_2 R_m M'_1 \cdot M'_2 : [c_2/\alpha] \sigma'} \\
\frac{\Gamma \vdash_{\kappa} M_1 R_m M'_1 : \sigma_1 \quad \Gamma \vdash_{\kappa} M_2 R_m M'_2 : \sigma_2}{\Gamma \vdash_{\kappa} \langle M_1, M_2 \rangle R_m \langle M'_1, M'_2 \rangle : \sigma_1 \times \sigma_2} \\
\frac{\Gamma \vdash_{\text{P}} M R_m M' : \Sigma \alpha : \sigma_1 . \sigma_2 \quad \Gamma \vdash_{\text{P}} M R_m M' : \Sigma \alpha : \sigma_1 . \sigma_2 \quad \Gamma \vdash \text{Fst}(M) \gg c}{\Gamma \vdash_{\text{P}} \pi_1 M R_m \pi_1 M' : \sigma_1 \quad \Gamma \vdash_{\text{P}} \pi_2 M R_m \pi_2 M' : [\pi_1 c/\alpha] \sigma_2} \\
\frac{\Gamma \vdash e R_t e' : \exists \alpha : k . \tau \quad \Gamma, \alpha : k, x : \tau \vdash_1 M R_m M' : \sigma \quad \Gamma \vdash \sigma : \text{sig}}{\Gamma \vdash_1 \text{unpack} [\alpha, x] = e \text{ in } (M : \sigma) R_m \text{unpack} [\alpha, x] = e' \text{ in } (M' : \sigma) : \sigma} \\
\frac{\Gamma \vdash e R_t e' : \tau \quad \Gamma, x : \tau \vdash_{\kappa} M R_m M' : \sigma}{\Gamma \vdash_{\kappa} \text{let } x = e \text{ in } M R_m \text{let } x = e' \text{ in } M' : \sigma} \\
\frac{\Gamma \vdash_1 M_1 R_m M'_1 : \sigma \quad \Gamma, \alpha / m : \sigma \vdash_1 M_2 R_m M'_2 : \sigma' \quad \Gamma \vdash \sigma' : \text{sig}}{\Gamma \vdash_1 \text{let } \alpha / m = M_1 \text{ in } (M_2 : \sigma') R_m \text{let } \alpha / m = M'_1 \text{ in } (M'_2 : \sigma') : \sigma'} \\
\frac{\Gamma \vdash_1 M R_m M' : \sigma}{\Gamma \vdash_1 (M :> \sigma) R_m (M' :> \sigma) : \sigma} \quad \frac{\Gamma \vdash_{\text{P}} M R_m M' : \sigma}{\Gamma \vdash_1 M R_m M' : \sigma} \quad \frac{\Gamma \vdash_{\kappa} M R_m M' : \sigma \quad \Gamma \vdash \sigma \leq \sigma'}{\Gamma \vdash_{\kappa} M R_m M' : \sigma'}
\end{array}$$

REFERENCES

- Val Breazu-Tannen, Thierry Coquand, Carl A. Gunter, and Andre Scedrov. 1991. Inheritance as Implicit Coercion. *Information and Computation* 93 (1991), 172–221.
- Karl Crary. 2017. Modules, Abstraction, and Parametric Polymorphism. In *Forty-Fourth ACM Symposium on Principles of Programming Languages*. Paris, France.
- Karl Crary, Robert Harper, and Sidd Puri. 1999. What is a Recursive Module?. In *1999 SIGPLAN Conference on Programming Language Design and Implementation*. Atlanta, 50–63.
- Pierre-Louis Curien and Giorgio Ghelli. 1992. Coherence of Subsumption, Minimum Typing and Type-Checking in $F_{\leq}$. *Mathematical Structures in Computer Science* 2, 1 (1992), 55–91.
- Derek Dreyer. 2005. *Understanding and Evolving the ML Module System*. Ph.D. Dissertation. Carnegie Mellon University, School of Computer Science, Pittsburgh, Pennsylvania.
- Derek Dreyer, Karl Crary, and Robert Harper. 2003. A Type System for Higher-Order Modules. In *Thirtieth ACM Symposium on Principles of Programming Languages*. New Orleans, Louisiana, 236–249.
- Martin Elmsan. 1999. Static Interpretation of Modules. In *1999 ACM International Conference on Functional Programming*. Paris, France, 208–219.
- Martin Elmsan, Troels Henriksen, Danil Annenkov, and Cosmin E. Oancea. 2018. Static Interpretation of Higher-Order Modules in Futhark: Functional GPU Programming in the Large. In *2018 ACM International Conference on Functional Programming*.
- Robert Harper and Mark Lillibridge. 1994. A Type-Theoretic Approach to Higher-Order Modules with Sharing. In *Twenty-First ACM Symposium on Principles of Programming Languages*. Portland, Oregon, 123–137.
- Robert Harper, David MacQueen, and Robin Milner. 1986. *Standard ML*. Technical Report ECS-LFCS-86-2. Department of Computer Science, University of Edinburgh.
- Robert Harper and John C. Mitchell. 1993. On the Type Structure of Standard ML. *ACM Transactions on Programming Languages and Systems* 15, 2 (April 1993), 211–252.
- Robert Harper, John C. Mitchell, and Eugenio Moggi. 1990. Higher-Order Modules and the Phase Distinction. In *Seventeenth ACM Symposium on Principles of Programming Languages*. San Francisco, 341–354.
- Robert Harper and Greg Morrisett. 1995. Compiling Polymorphism Using Intensional Type Analysis. In *Twenty-Second ACM Symposium on Principles of Programming Languages*. San Francisco, 130–141.
- Robert Harper and Benjamin C. Pierce. 2005. Design Considerations for ML-Style Module Systems. In *Advanced Topics in Types and Programming Languages*, Benjamin C. Pierce (Ed.). The MIT Press.
- Robert Harper and Chris Stone. 2000. A Type-Theoretic Interpretation of Standard ML. In *Proof, Language and Interaction: Essays in Honour of Robin Milner*. The MIT Press. Extended version published as CMU technical report CMU-CS-97-147.
- Daniel K. Lee, Karl Crary, and Robert Harper. 2007. Towards a Mechanized Metatheory of Standard ML. In *Thirty-Fourth ACM Symposium on Principles of Programming Languages*. Nice, France.
- Xavier Leroy. 1994. Manifest Types, Modules and Separate Compilation. In *Twenty-First ACM Symposium on Principles of Programming Languages*. Portland, Oregon, 109–122.

- Xavier Leroy. 1995. Applicative Functors and Fully Transparent Higher-Order Modules. In *Twenty-Second ACM Symposium on Principles of Programming Languages*. San Francisco.
- Xavier Leroy. 2003. A proposal for recursive modules in Objective Caml. (2003). Available at http://caml.inria.fr/pub/papers/xleroy-recursive_modules-03.pdf.
- Xavier Leroy, Damien Doligez, Jacques Garrigue, Didier Rémy, and Jérôme Vouillon. 2016. *The OCaml system, release 4.03, Documentation and user's manual*. Institut National de Recherche en Informatique et Automatique (INRIA).
- David MacQueen. 1986. Using Dependent Types to Express Modular Structure. In *Thirteenth ACM Symposium on Principles of Programming Languages*. St. Petersburg Beach, Florida, 277–286.
- David B. MacQueen and Mads Tofte. 1994. A Semantics for Higher-order Functors. In *Fifth European Symposium on Programming (Lecture Notes in Computer Science)*, Vol. 788. Springer, 409–423.
- Robin Milner, Mads Tofte, and Robert Harper. 1990. *The Definition of Standard ML*. The MIT Press, Cambridge, Massachusetts.
- John C. Mitchell and Gordon D. Plotkin. 1988. Abstract Types Have Existential Type. *ACM Transactions on Programming Languages and Systems* 10, 3 (July 1988), 470–502.
- Marco Patrignani, Amal Ahmed, and Dave Clarke. 2019. Formal Approaches to Secure Compilation. *Comput. Surveys* (2019). To appear.
- Andrew Pitts. 2005. Typed Operational Reasoning. In *Advanced Topics in Types and Programming Languages*, Benjamin C. Pierce (Ed.). The MIT Press, Chapter 7, 245–289.
- Andreas Rossberg, Claudio Russo, and Derek Dreyer. 2014. F-ing Modules. *Journal of Functional Programming* 24, 5 (Sept. 2014).
- Claudio V. Russo. 1998. *Types for Modules*. Ph.D. Dissertation. Edinburgh University.
- Claudio V. Russo. 2000. First-Class Structures for Standard ML. *Nordic Journal of Computing* 7, 4 (2000).
- Claudio V. Russo. 2001. Recursive Structures for Standard ML. In *2001 ACM International Conference on Functional Programming*.
- Chung-chieh Shan. 2006. Higher-order modules in System F_{ω} and Haskell. (May 2006). Available at homes.soic.indiana.edu/ccshan/xlate/xlate.pdf.
- Zhong Shao. 1998. Typed Cross-Module Compilation. In *1998 ACM International Conference on Functional Programming*. Baltimore, Maryland, 141–152.
- Zhong Shao. 1999. Transparent Modules with Fully Syntactic Signatures. In *1999 ACM International Conference on Functional Programming*. Paris, 220–232.
- Christopher A. Stone and Robert Harper. 2000. Deciding Type Equivalence in a Language with Singleton Kinds. In *Twenty-Seventh ACM Symposium on Principles of Programming Languages*. Boston. Extended version published as CMU technical report CMU-CS-99-155.
- Christopher A. Stone and Robert Harper. 2006. Extensional Equivalence and Singleton Types. *ACM Transactions on Computational Logic* 7, 4 (Oct. 2006). An earlier version appeared in the 2000 Symposium on Principles of Programming Languages.